SAMSUNG

Samsung Innovation Campus

Vạn vật kết nối – IoT

Together for Tomorrow!
Enabling People
Education for Future Generations

Chương 7.

Node.js trong IoT

Vạn vật kết nối – IoT

Mô tả chương

Mục tiêu của chương

- Tìm hiểu về khái niệm cơ bản về Node.js, cú pháp lập trình và các kỹ thuật lập trình nâng cao hơn.
- Đồng thời học cách điều khiển các cảm biến khác nhau của Raspberry Pi trên Node.js.

Nội dung của chương

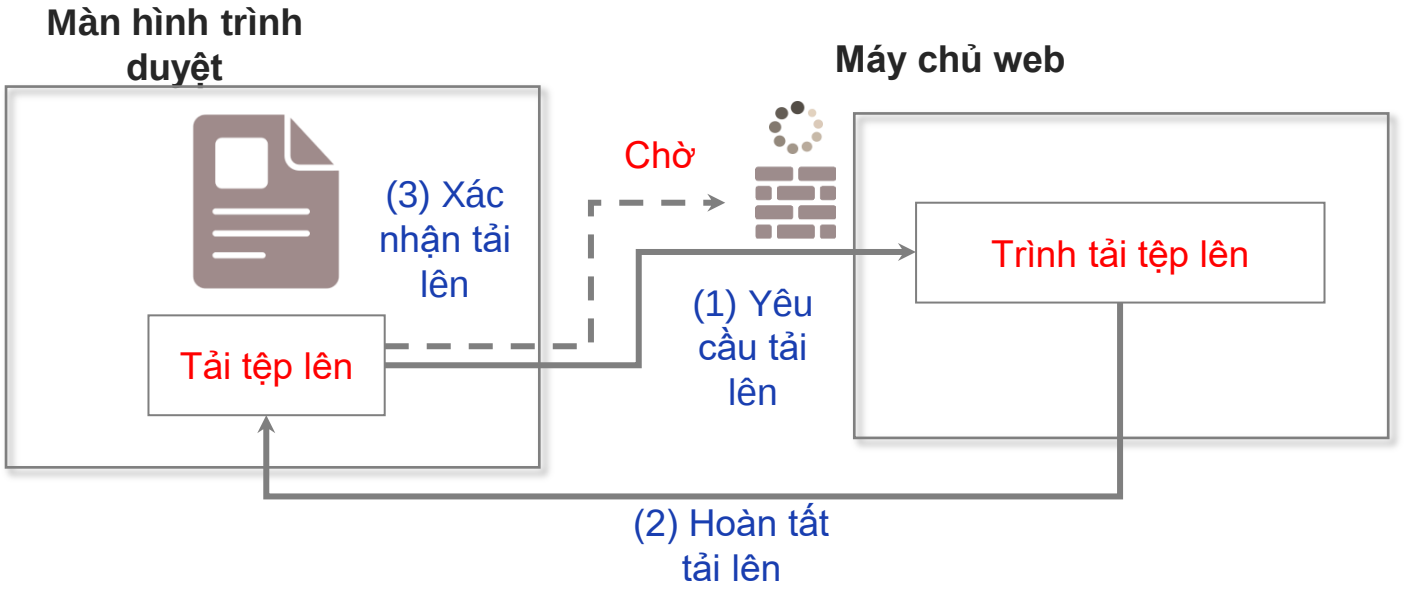
- Bài 1. Tổng quan về Node.js
- Bài 2. Lập trình Node.js cơ bản
- Bài 3. Raspberry Pi với Node-RED
- Bài 4. Xử lý lỗi và khắc phục sự cố

Bài 1.

Tổng quan về Node.js

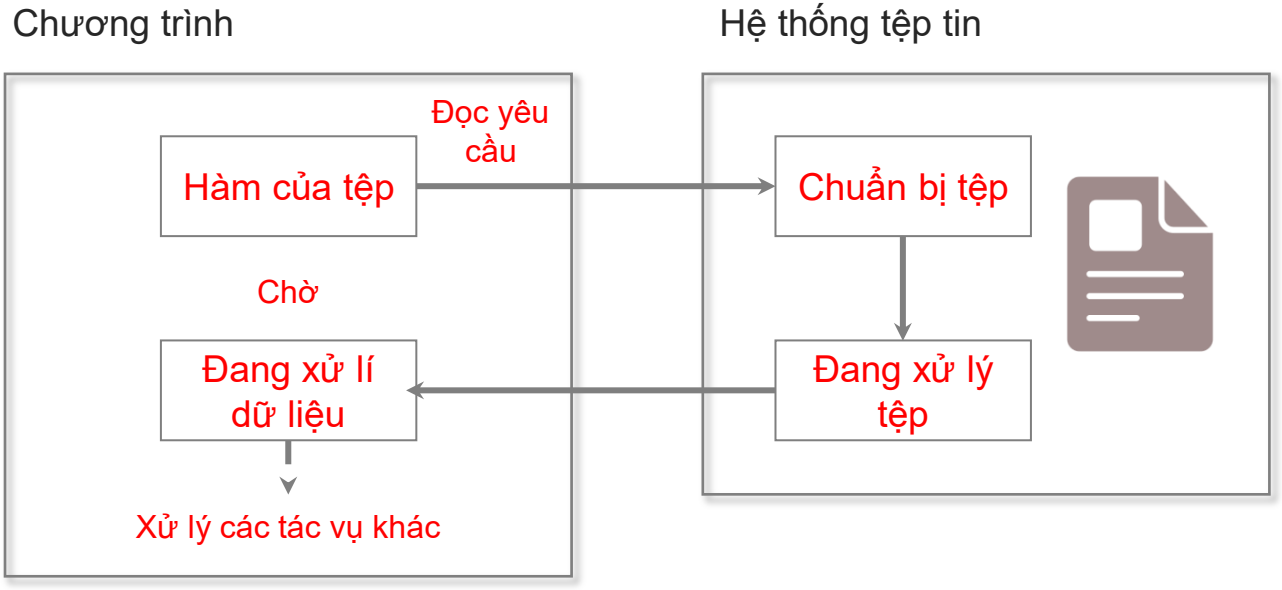
- 1.1. Tổng quan về Node.js
- 1.2. Cài đặt các công cụ phát triển
- 1.3. Tạo dự án node đầu tiên
- 1.4. Gửi log lên console
- 1.5. Sử dụng Mô-đun trong Node
- 1.6. Sử dụng mô-đun tích hợp sẵn

- Node.js còn được gọi là Node
- Hiện đã có nhiều ngôn ngữ và công cụ giúp xây dựng các hàm của máy chủ như các dịch vụ Web. Tại sao Node được tạo ra?
  - Bắt đầu xây dựng các công cụ máy chủ mới để giải quyết các vấn đề xảy ra trong quá trình tải tệp lên.



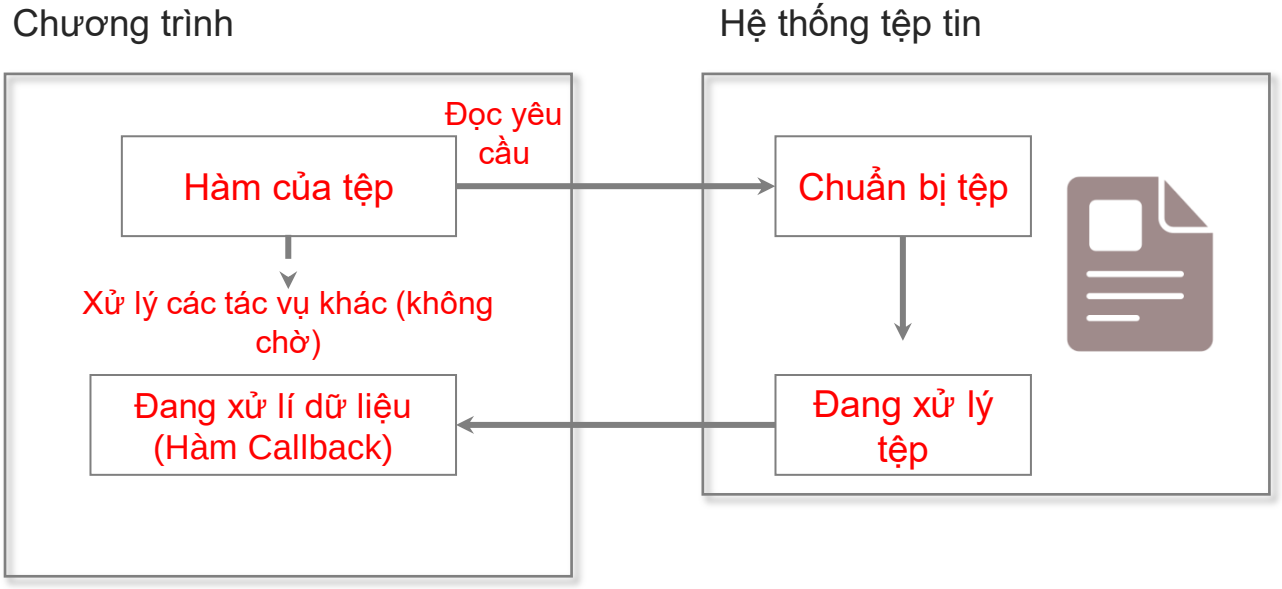
Các phương thức xử lý I/O đồng bộ trong Node

- Các phương thức xử lý I/O đồng bộ trong Node
  - Xử lý các tác vụ khác sau khi đọc nội dung của tệp
  - Dưới đây là ví dụ về I/O đồng bộ:



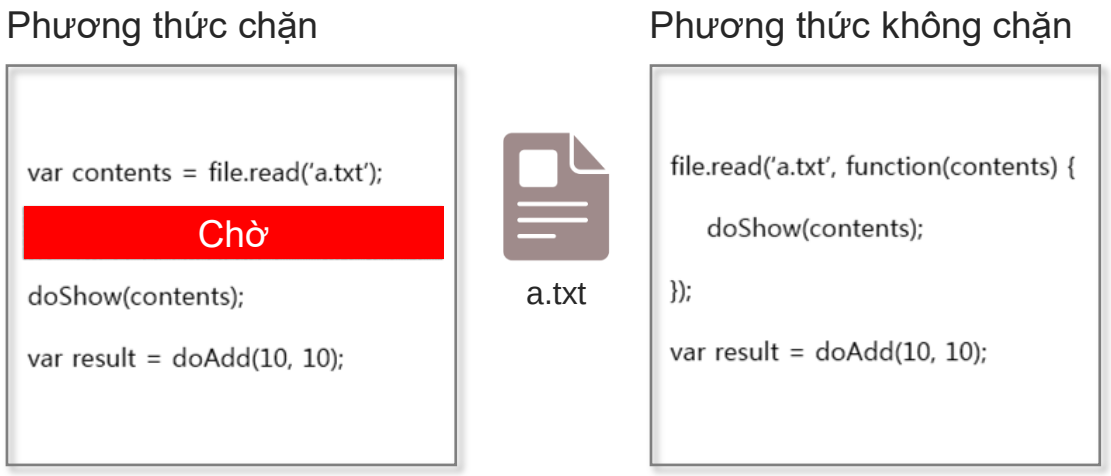
Các phương thức I/O không đồng bộ trong Node

- Các phương thức I/O đồng bộ trong Node
  - Xử lý đồng thời các yêu cầu khác mà không cần đợi quá trình yêu cầu đơn lẻ kết thúc
  - → I/O không chặn
  - Dưới đây là một ví dụ về I/O không đồng bộ có sử dụng hàm Callback



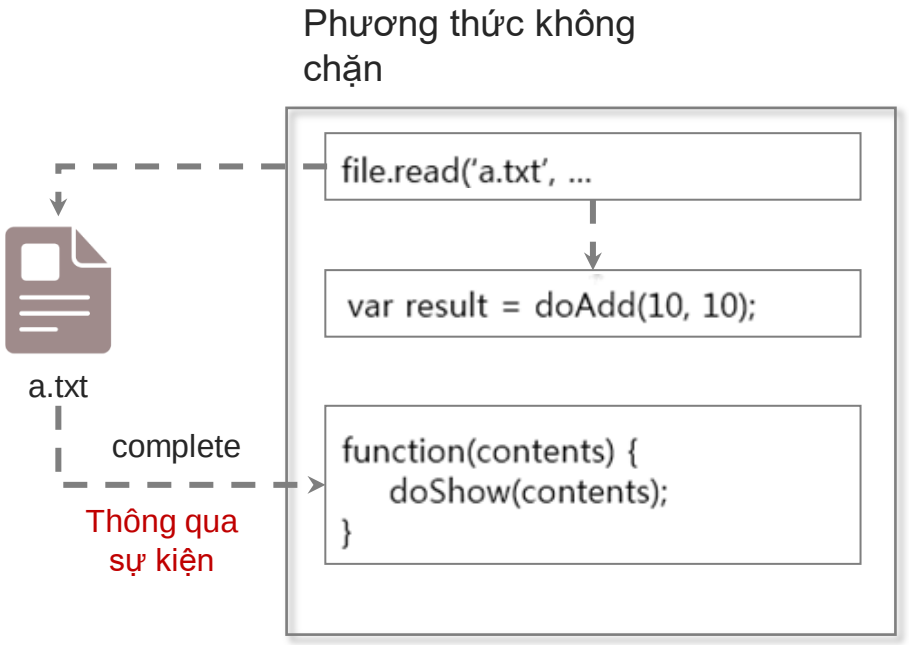
Đồng bộ và Không đồng bộ, so sánh mẫu code

- Phương thức đồng bộ đọc nội dung của tệp và trả về dưới dạng giá trị kết quả của hàm. Chờ đến khi kết quả được trả về.
- Phương thức không đồng bộ đọc nội dung tệp và trả về dưới dạng hàm Callback. Lệnh thực thi tiếp theo sẽ được tiến hành ngay sau khi tệp bắt đầu được đọc



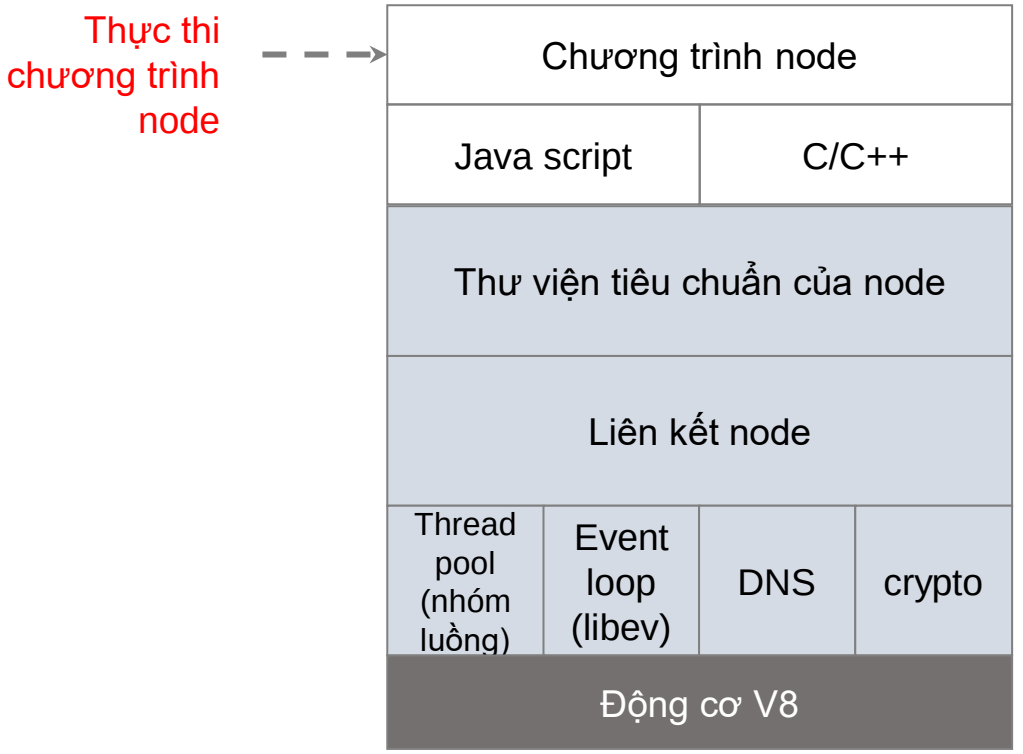
Phương thức I/O dựa trên sự kiện

- Phương thức không đồng bộ còn được gọi là mô hình I/O dựa trên sự kiện
- Hàm Callback được thực thi khi hệ thống nhận được sự kiện



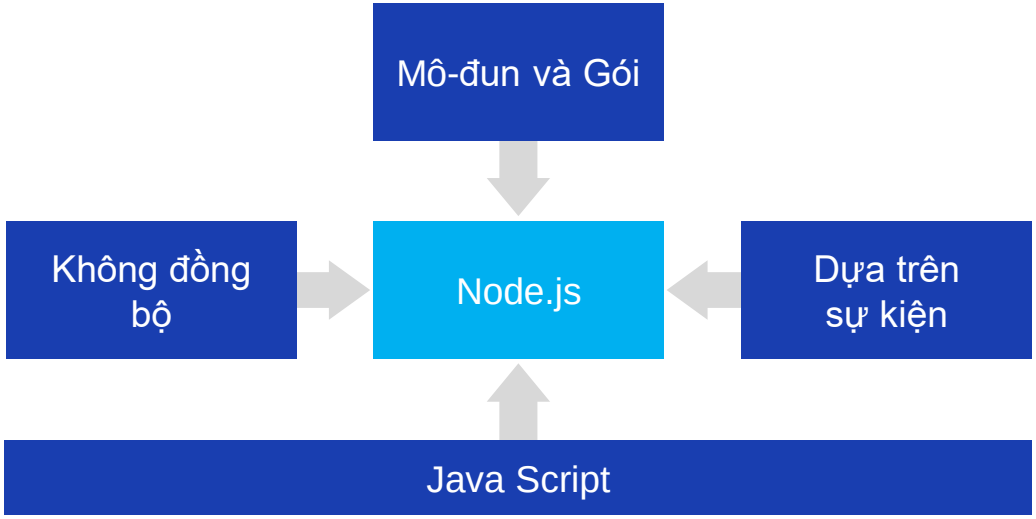
Kiến trúc Node

- Chạy trên V8 Engine của trình duyệt Google Chrome và sử dụng ngôn ngữ Javascript



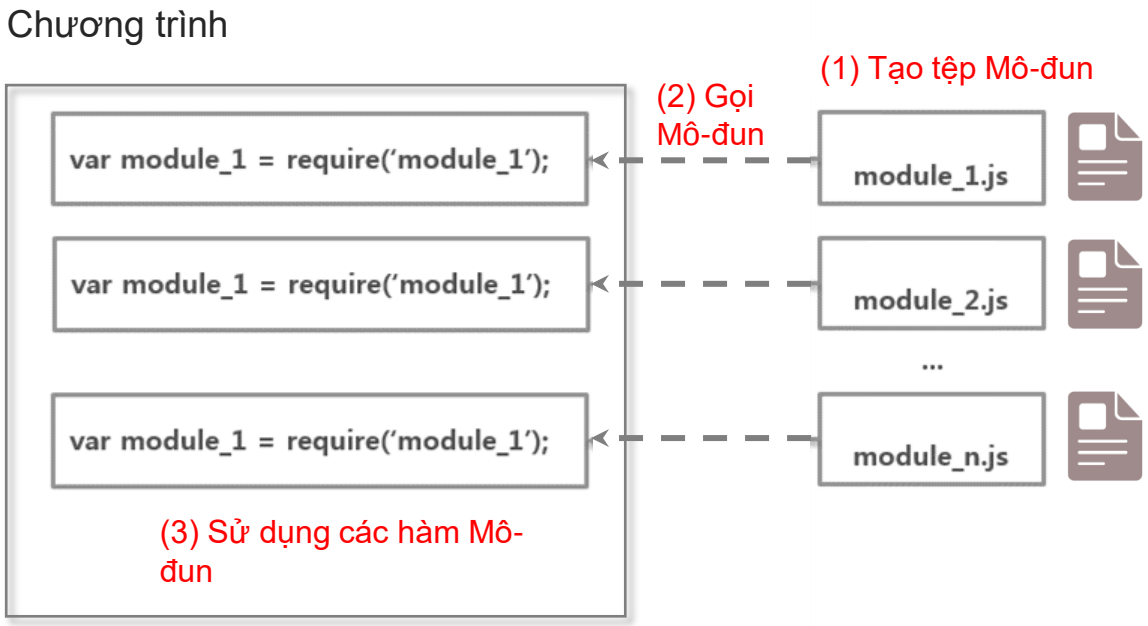
Các tính năng chính của Node

- JavaScript là một ngôn ngữ lập trình đã được nhiều nhà phát triển web sử dụng.
- Sử dụng I/O không đồng bộ-/I/O dựa trên sự kiện
- Cấu hình các chương trình máy chủ bằng các mô-đun và gói
- Các nhà phát triển đã tự phát triển rất nhiều mô-đun



Các mô-đun giúp các node dễ sử dụng hơn

- Tách mã JavaScript thành một Mô-đun, một tệp riêng biệt có phần mở rộng là js
- **Tuân thủ thông số tiêu chuẩn CommonJs**



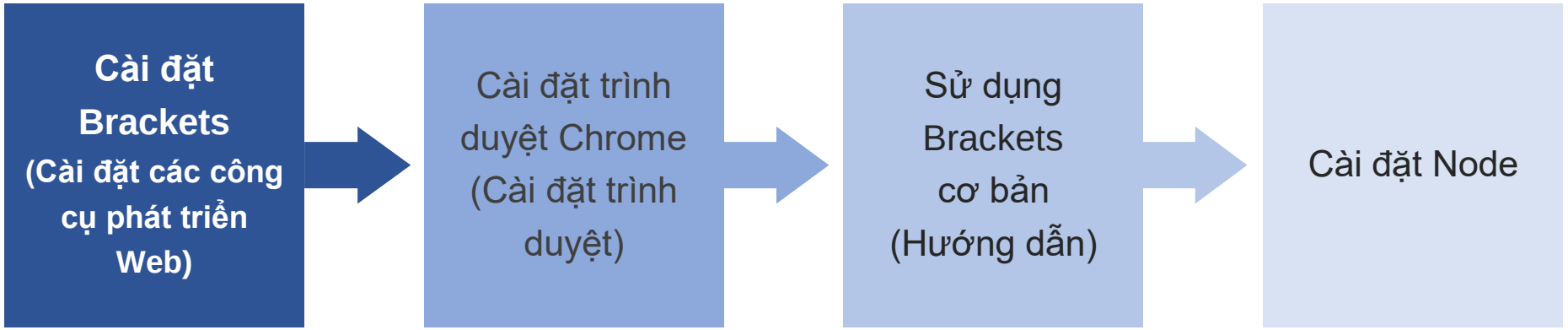
Bài 1.

Tổng quan về Node.js

- 1.1. Tổng quan về Node.js
- 1.2. Cài đặt các công cụ phát triển
- 1.3. Tạo dự án node đầu tiên
- 1.4. Gửi log lên console
- 1.5. Sử dụng Mô-đun trong Node
- 1.6. Sử dụng mô-đun tích hợp sẵn

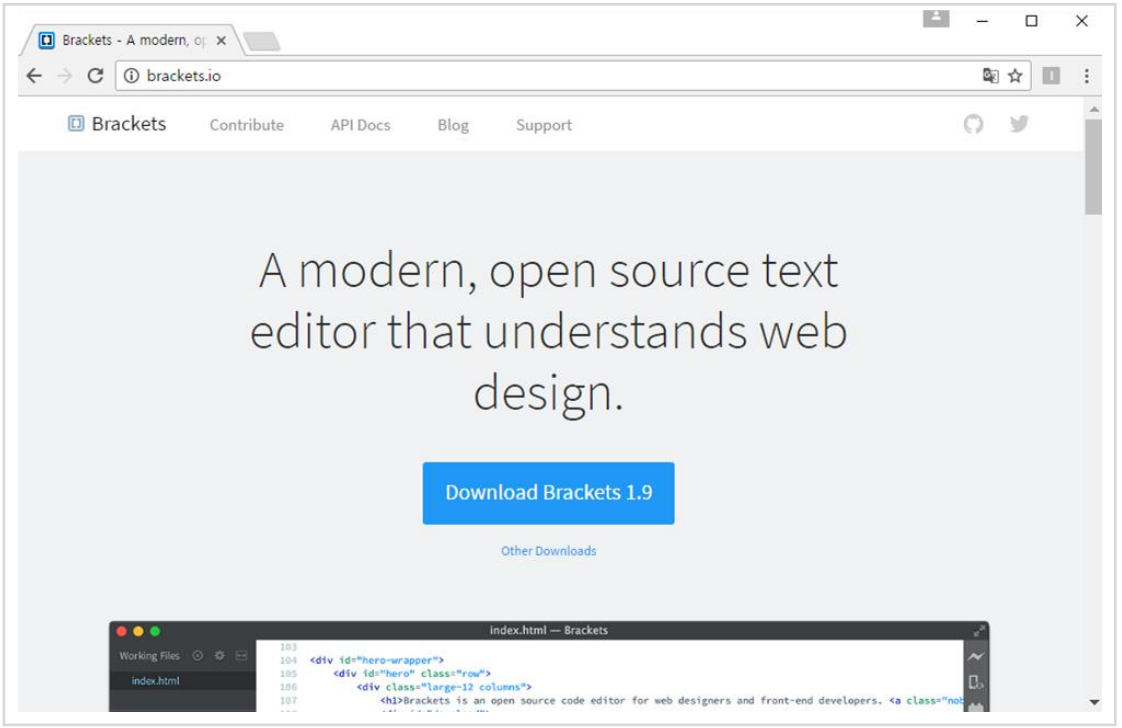
Tiến trình cài đặt

- Có thể tạo mã nguồn của node bằng một trình soạn thảo đơn giản → Brackets, EditPlus, v.v.
- Khóa học này sử dụng trình soạn thảo văn bản nguồn Brackets



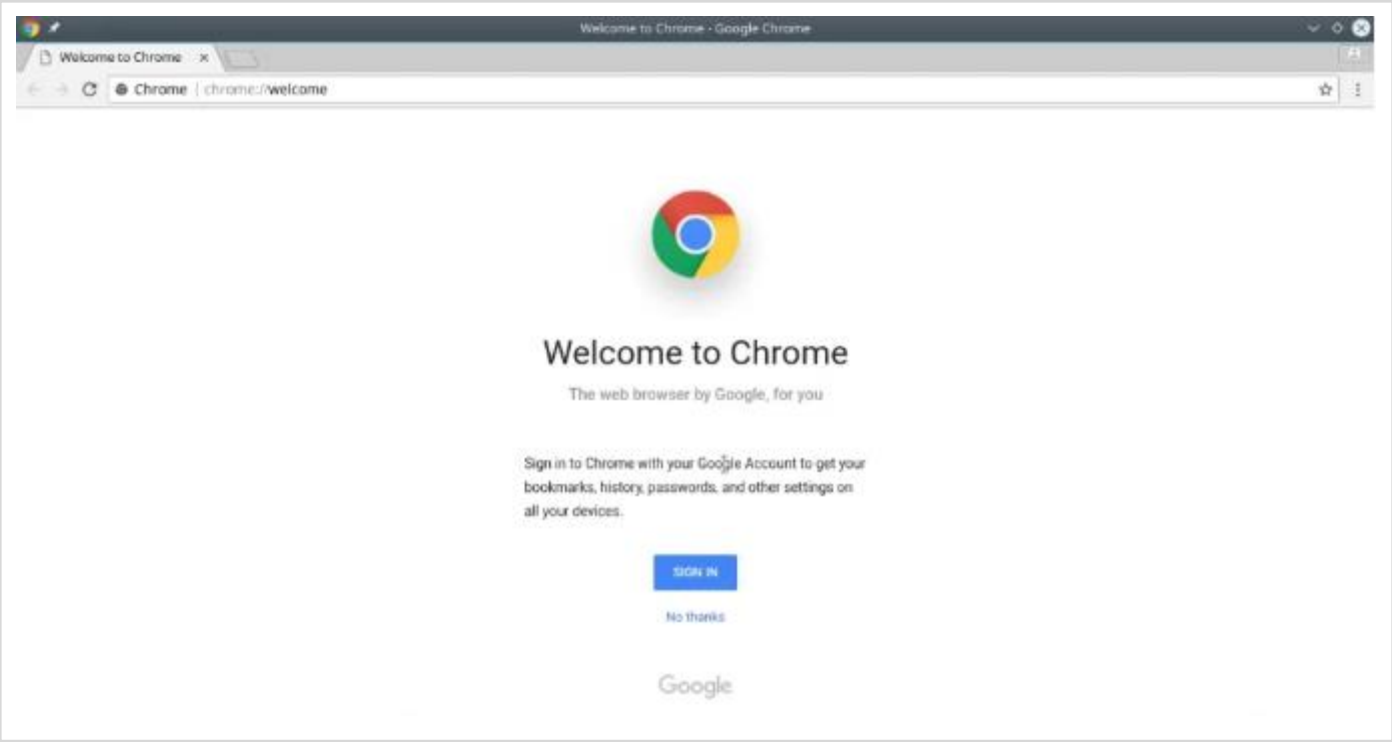
Cài đặt trình soạn thảo văn bản nguồn Brackets

- Tải xuống từ http://brackets.io và cài đặt



Cài đặt trình duyệt Chrome

- Tải xuống từ http://www.google.com và cài đặt



Cài đặt Node.js

- Có phiên bản LTS và phiên bản Current. Phiên bản LTS là phiên bản duy trì các hàm hiện có trong một khoảng thời gian nhất định để ổn định hệ thống và Phiên bản Current là phiên bản mới nhất.
- Khóa học này sử dụng phiên bản Current.



Thay đổi chủ đề của Brackets

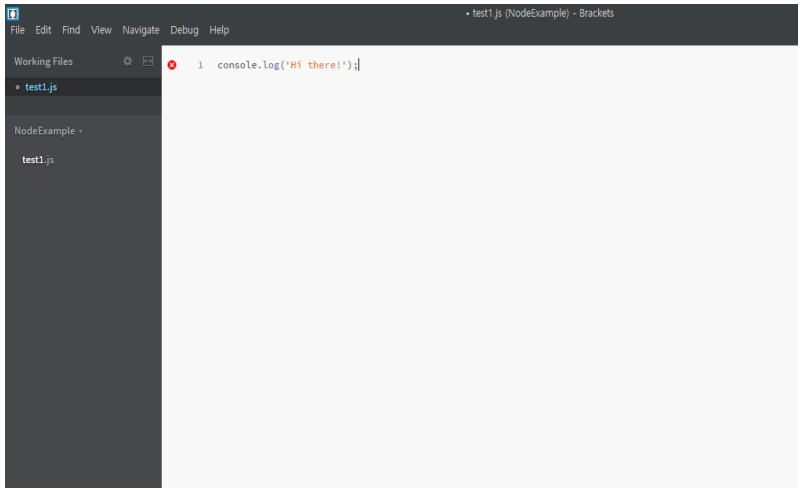
- Có thể thay đổi chủ đề của brackets: Nhấp vào View → menu Themes
- Tăng kích cỡ phông chữ: <Ctrl> + +
- Giảm kích cỡ phông chữ: <Ctrl> + -

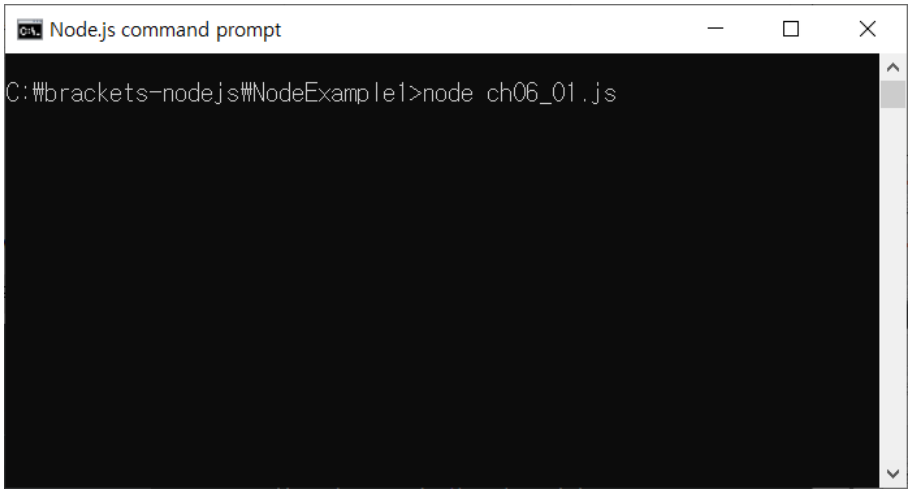
Mở thư mục Project

- Trong File Explorer, tạo thư mục “brackets_nodejs” tại thư mục “NodeExample” nằm trong thư mục tài khoản người dùng Windows
- Nhấp vào File → Mở menu Folder để chỉ định vị trí của thư mục
- Nhấp chuột phải vào vùng bên trái của dự án và nhấp vào menu New File để tạo tệp

Tạo một tệp JavaScript và chạy trong dấu nhắc lệnh

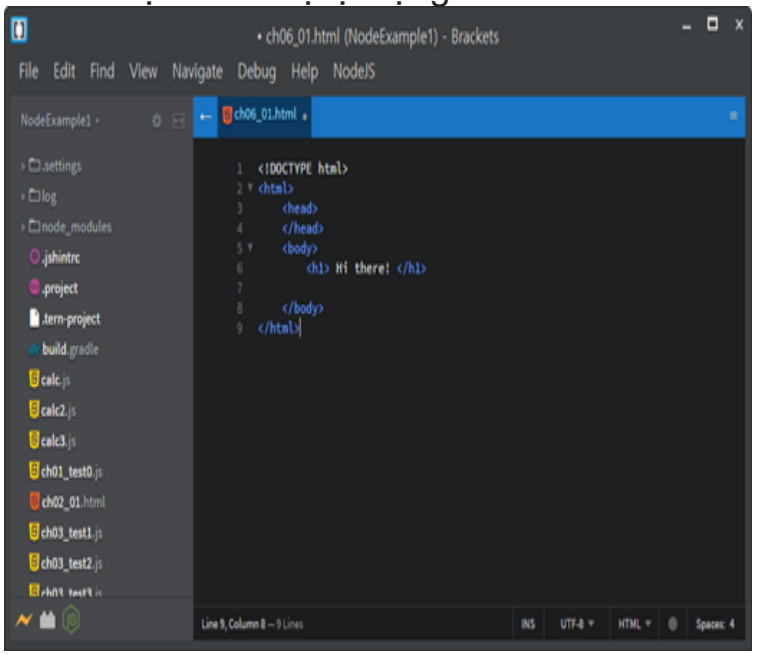
- Nhập một mã JavaScript đơn giản
- Đi tới thư mục dự án và chạy tệp tại node test1.js trong dấu nhắc lệnh
  (Nếu đã tạo tên tệp là ch06_01.js, hãy chạy tệp này dưới dạng node ch06_01.js)

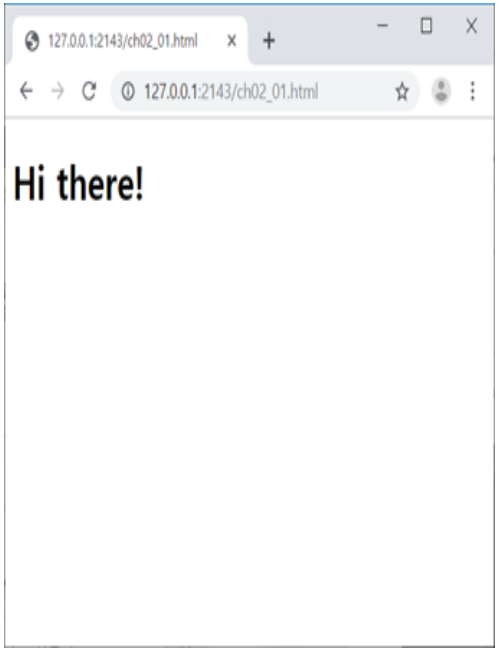




Tạo tệp HTML và xem trước trong trình duyệt web

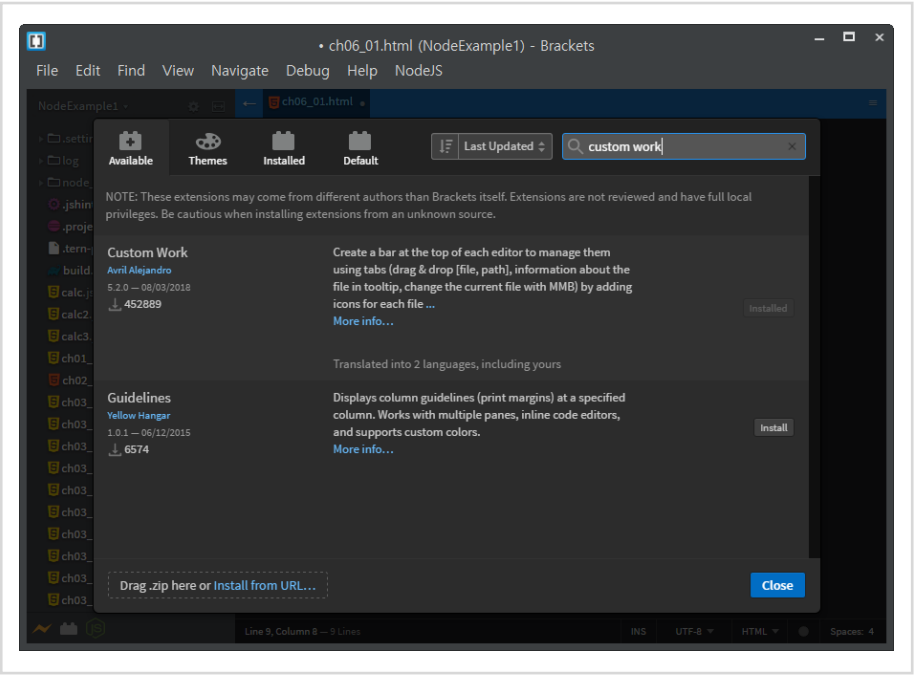
- Tạo tệp ch06_01.html và nhập các thẻ HTML đơn giản
- Nhấp vào biểu tượng xem trước trực tiếp để mở trang trong trình duyệt Chrome
- Các mã thay đổi sẽ được hiển thị tự động





Cài đặt phần mở rộng

- Có sẵn nhiều phần mở rộng
- Nhấp vào biểu tượng [Extension Manager] và cài đặt menu plug-in (phần bổ trợ) bên dưới:
- Brackets Icon Custom Work (Nếu bạn thấy hộp thoại Option sau khi cài đặt, hãy tắt Use Icon Option ở trên cùng)



Bài 1.

Tổng quan về Node.js

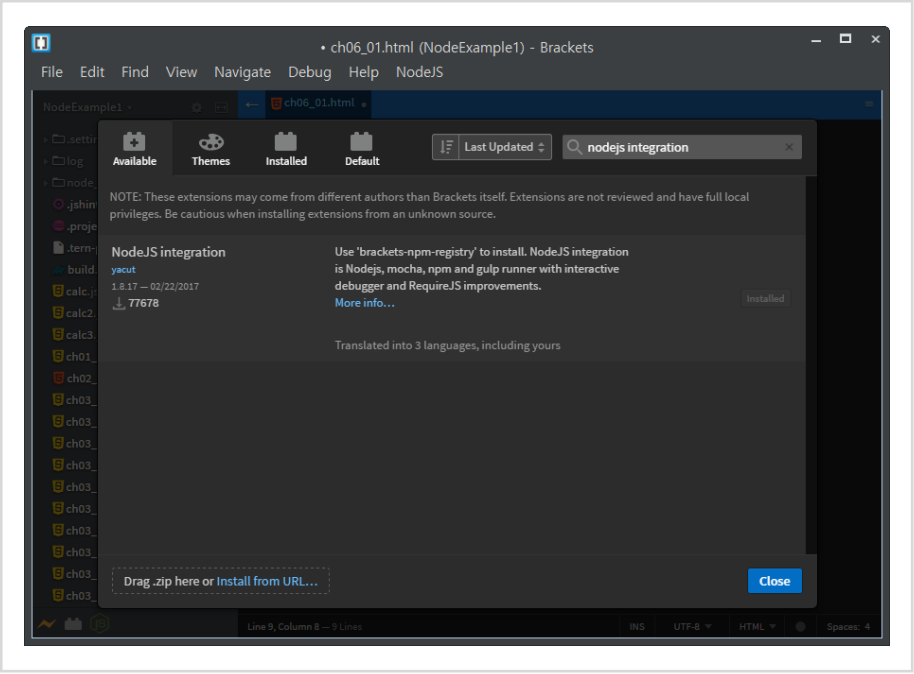
- 1.1. Tổng quan về Node.js
- 1.2. Cài đặt các công cụ phát triển
- 1.3. Tạo dự án node đầu tiên
- 1.4. Gửi log lên console
- 1.5. Sử dụng Mô-đun trong Node
- 1.6. Sử dụng mô-đun tích hợp sẵn

Tạo node trong Brackets

- Nhấp vào File → Mở menu Folder và chỉ định thư mục NodeExample trong thư mục brackets_nodejs

  (Nếu đã được chỉ định sẵn, hãy giữ nguyên)

- Cài đặt NodeJs integration trong số các hàm mở rộng



Chạy node trực tiếp trong Brackets

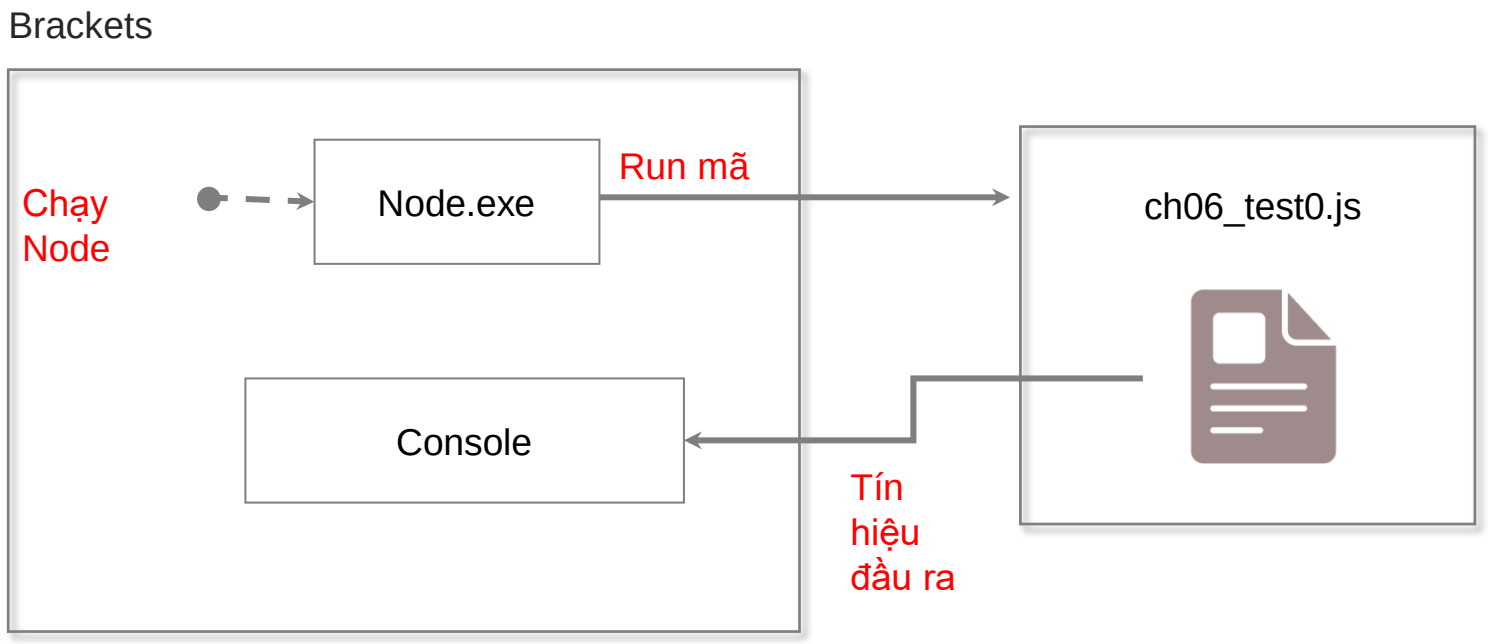
- Tệp JavaScript có thể được thực thi bằng cách mở dấu nhắc lệnh và sử dụng node có thể thực thi được.
- Có thể chạy bằng phần mở rộng của NodeJs integration trong Brackets
- Chọn tệp trong vùng bên trái của dự án và nhấn chuột phải để chọn menu Add to Node.js.
- Chọn tệp mới tạo từ thẻ dưới cùng và chạy tệp đó

Gỡ lỗi các node trong Brackets

- Gỡ lỗi bằng nút gỡ lỗi

Thực thi chương trình Node

- Khi sử dụng lệnh node, nội dung của tệp JavaScript chỉ định sẽ được đọc và thực thi. Kết quả thực thi sẽ
- được hiển thị trong console.



Bài 1.

Tổng quan về Node.js

- 1.1. Tổng quan về Node.js
- 1.2. Cài đặt các công cụ phát triển
- 1.3. Tạo dự án node đầu tiên
- 1.4. Gửi log lên console
- 1.5. Sử dụng Mô-đun trong Node
- 1.6. Sử dụng mô-đun tích hợp sẵn

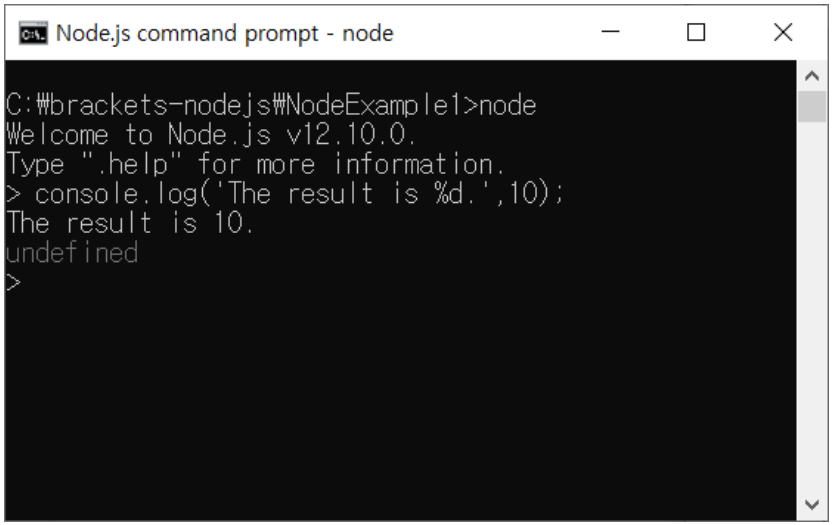
Chạy các tệp JavaScript trong dấu nhắc lệnh

- Khi bạn chạy dấu nhắc lệnh có lệnh cmd, thư mục người dùng sẽ được đặt làm vị trí mặc định.
- Chuyển đến thư mục NodeExample và chạy javascript bằng lệnh node (không cần nhập ký hiệu %).

```
% cd brackets_nodejs\NodeExample
% node ch06_test1.js
```

Viết và thực thi mã ở dấu nhắc lệnh

- Có thể nhập mã trực tiếp ở dấu nhắc lệnh
- Khi bạn thực thi lệnh node, hệ thống cho phép bạn bắt đầu nhập mã.




```
% node
% console.log('The result is %d.', 10);
```

Đối tượng Console

- Console hiển thị các log dưới dạng các đối tượng toàn cục có thể được sử dụng ở mọi nơi

Tên đối tượng toàn cục	Mô tả
console	Đối tượng hiển thị kết quả trong cửa sổ console
process	Đối tượng xử lý thông tin thực thi tiến trình
exports	Đối tượng xử lý mô-đun

```
% console.log('Display numbers : %d', 10);
% console.log('Display string : %s', 'Hello!');
% console.log('Display JSON Object : %j', {name: 'June'});
```

Các phương thức chính của các đối tượng Console

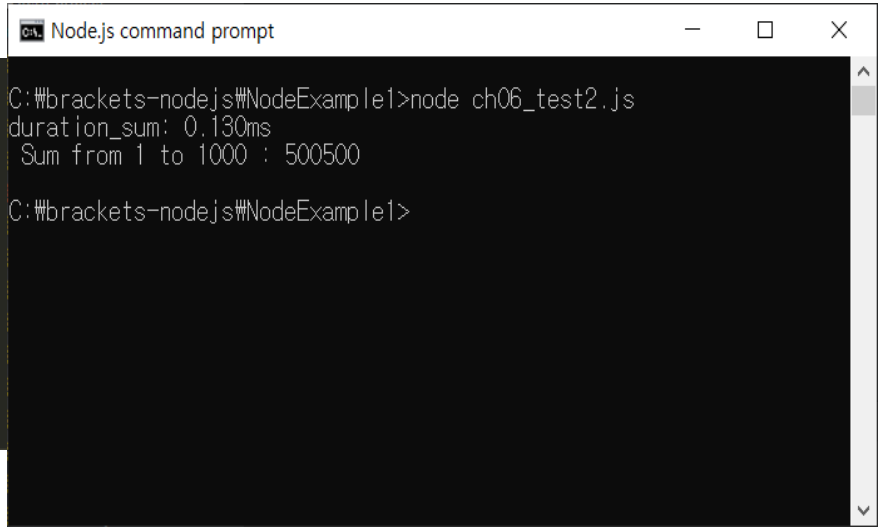
- Các phương thức như dir, time, timeEnd tồn tại trong các đối tượng Console

Tên đối tượng toàn cục	Mô tả
dir(object)	Hiển thị các thuộc tính của đối tượng JavaScript
time (id)	Ghi lại thời gian bắt đầu để đo thời gian chạy
timeEnd (id)	Ghi lại thời gian kết thúc để đo thời gian chạy

Kiểm tra thời gian thực thi mã

- Chọn dự án [NodeExample] ở bên trái, nhấp chuột phải và chọn [Create File].
- Nhập làm tên tệp
- Nhập mã sau vào tệp mới tạo và thực thi tệp

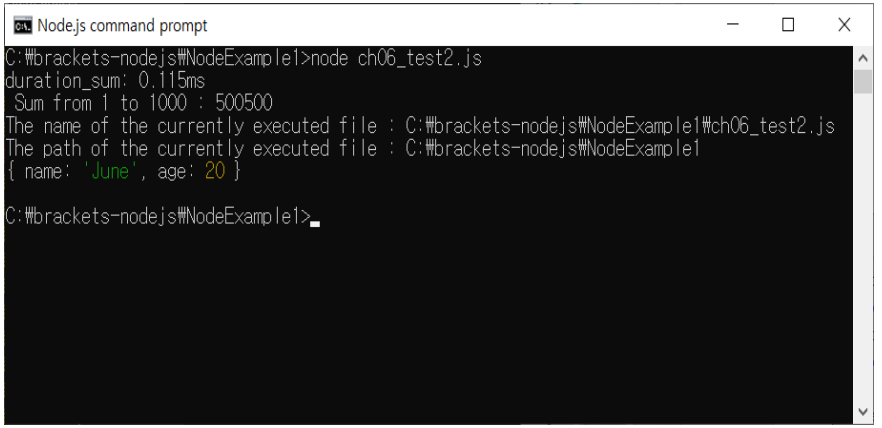
```
1  var result = 0;
2
3  console.time( label: 'duration_sum');
4  for (var i = 1; i <= 1000; i++) {
5      result += i;
6  }
7  console.timeEnd( label: 'duration_sum');
8  console.log(' Sum from 1 to 1000 : %d', result);
```



Hiển thị tên tệp và thông tin về đối tượng

- Hiển thị tên tệp bằng cách sử dụng các biến toàn cục như __filename, __dirname
- Hiển thị thông tin đối tượng bằng cách sử dụng hàm console.dir
- Tạo các đối tượng JavaScript bằng cách sử dụng {}
- Định dạng là {tên thuộc tính: giá trị thuộc tính, tên thuộc tính: giá trị thuộc tính}

```
1   var result = 0;
2
3   console.time( label: 'duration_sum');
4   for (var i = 1; i <= 1000; i++) {
5       result += i;
6   }
7   console.timeEnd( label: 'duration_sum');
8   console.log(' Sum from 1 to 1000 : %d', result);
9
10  console.log(' The name of the currently executed file : %s',
11      __filename);
12  console.log(' The path of the currently executed file : %s',
13      __dirname);
14
15  var Person = {name:"June", age:20};
16  console.dir(Person);
```



Đối tượng Process

- Đối tượng Process là một đối tượng xử lý thông tin về quy trình

Tên đối tượng toàn cục	Mô tả
argv	Thông tin về tham số được truyền đi khi thực thi tiến trình
env	Thông tin về biến môi trường
exit()	Phương thức kết thúc tiến trình

```
1  console.log(' Parameter number of argv attribute : '
2      + process.argv.length);
3  console.dir(process.argv);
4
5  process.argv.forEach(function(item, index) {
6      console.log(index + ' : ', item);
7  });
```

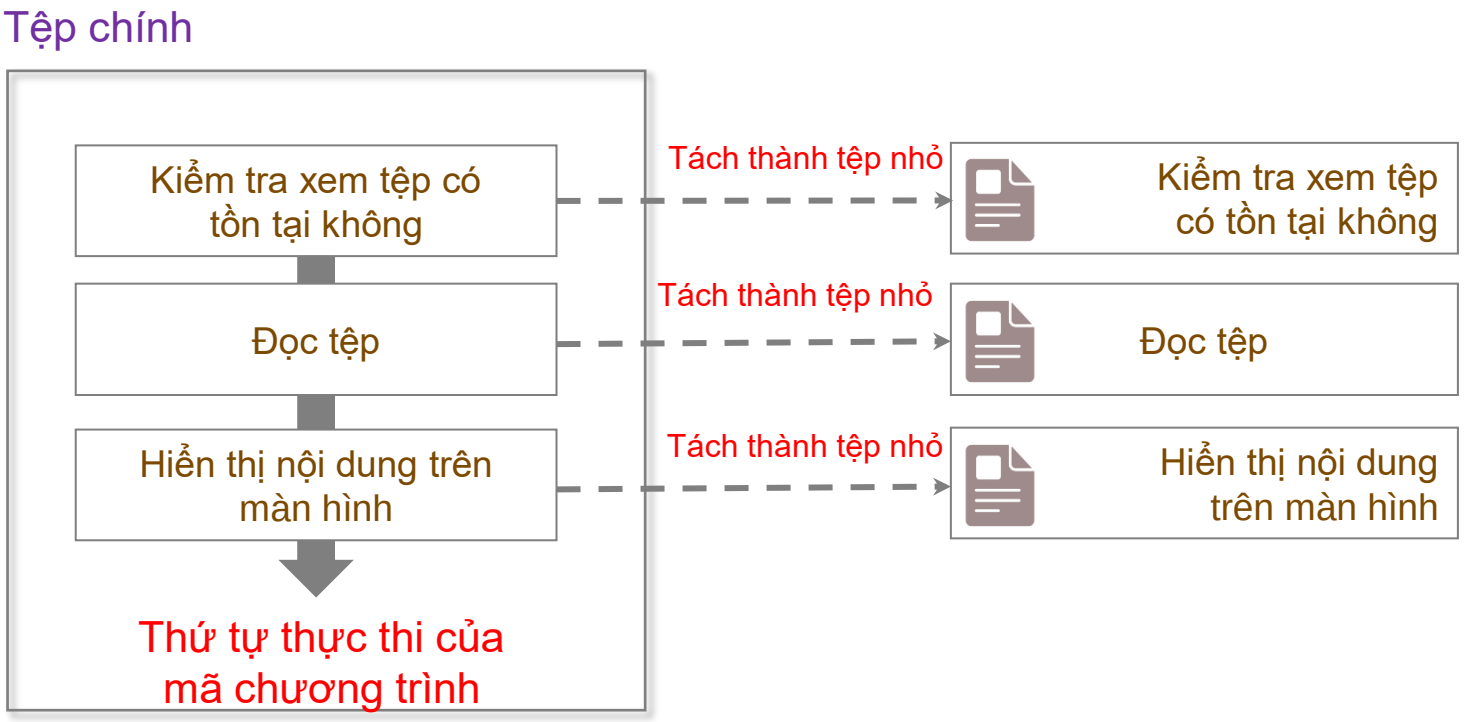
Bài 1.

Tổng quan về Node.js

- 1.1. Tổng quan về Node.js
- 1.2. Cài đặt các công cụ phát triển
- 1.3. Tạo dự án node đầu tiên
- 1.4. Gửi log lên console
- 1.5. Sử dụng Mô-đun trong Node
- 1.6. Sử dụng mô-đun tích hợp sẵn

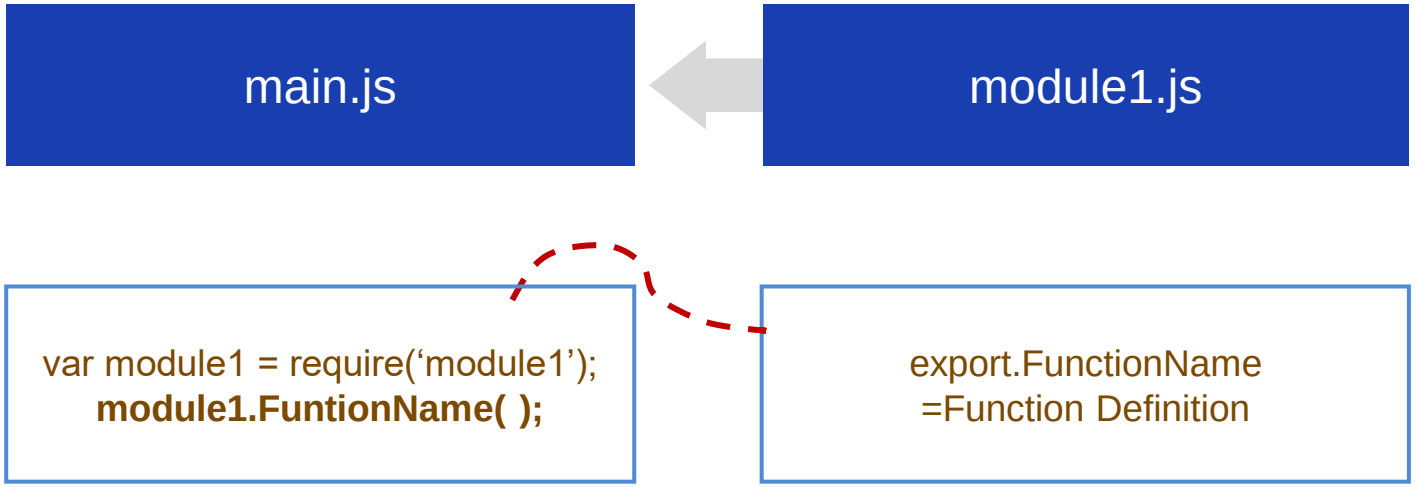
Chia hàm thành các tệp riêng biệt

- Có thể chia mã cần cho việc đọc tệp thành từng tệp tùy thuộc vào hàm



Mô-đun

- Mô-đun: Một hàm độc lập được tách thành một tệp
- Khi bạn đã tạo một mô-đun, bạn có thể tải mô-đun đó từ một tệp khác để sử dụng.
- Tuân thủ thông số tiêu chuẩn CommonJs và sử dụng đối tượng toàn cục exports



Sử dụng đối tượng exports và module.exports

- Bạn có thể sử dụng exports trong các tệp mô-đun hoặc module.exports
- Để trực tiếp phân bổ một đối tượng, hãy sử dụng module.exports.

module1.js

```
exports.add = function(a, b) {
    return a + b;
};
exports.multiply = function(a, b) {
    return a * b;
};
```

VS.

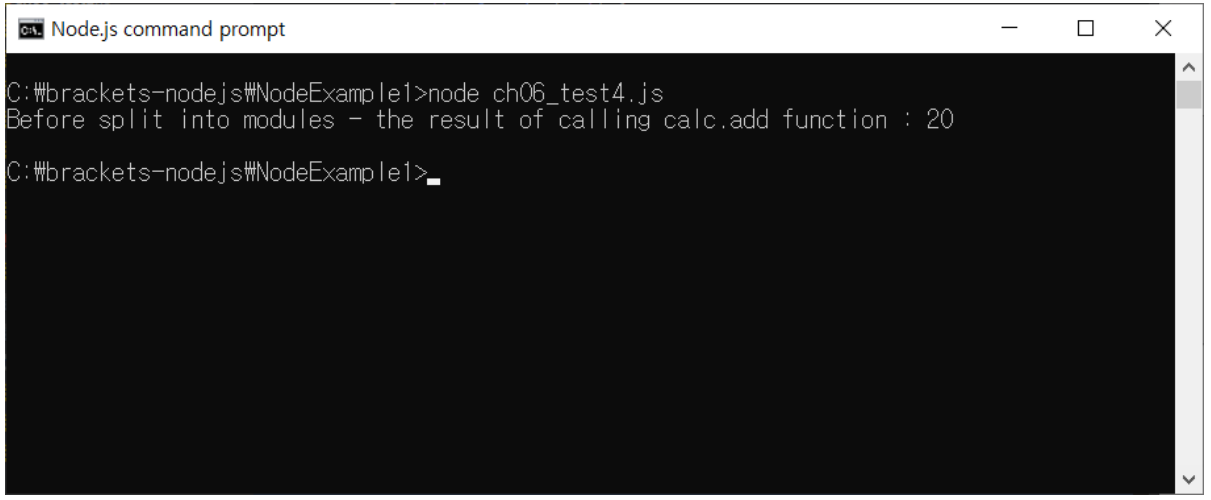
module2.js

```
var calc = {};
calc.add = function(a, b) {
    return a + b;
};
calc.multiply = function(a, b) {
    return a * b;
};
module.exports = calc;
```

Tách một hàm bổ sung thành các mô-đun

- Mã cho hàm plus trước khi tách thành các mô-đun

```
1   var calc = {};
2   calc.add = function(a, b) {
3       return a + b;
4   }
5   console.log(' Before splitting into modules - ' +
6       'the result of calling calc.add function : %d',
7       calc.add( a: 10,  b: 10));
```



Tệp mô-đun

- Tạo tệp mô-đun → calc.js
- Thêm một hàm dưới dạng thuộc tính add của đối tượng exports

```
1    exports.add = function(a, b) {
2        return a + b;
3    }
```

- Gọi hàm add sau khi nạp tệp mô-đun
- Nạp tệp mô-đun bằng hàm require
- Đối tượng result đã nhập có thể được coi như đối tượng exports
- Việc chỉ định một thư mục thay vì một tệp sẽ gọi tệp index.js trong thư mục đó

```
1   var calc = require('./calc');
2   console.log(' After splitting into modules - ' +
3       'the result of calling calc.add function : %d',
4       calc.add(10, 10));
```



Sử dụng đối tượng module.exports trong một tệp mô-đun

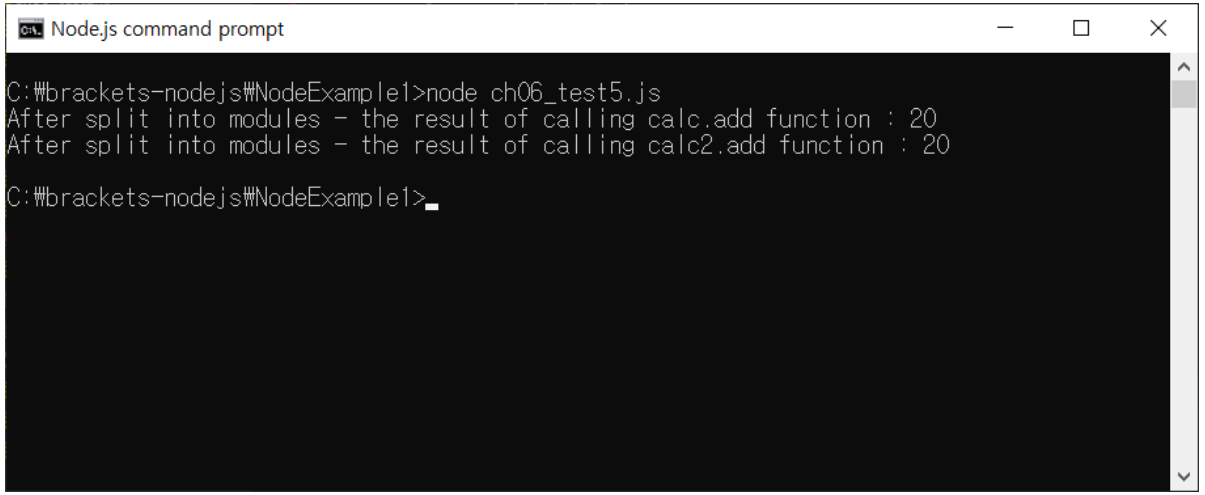
- Tạo tệp mô-đun, calc2.js
- Tạo một đối tượng calc và gán đối tượng đó cho module.exports

```
var calc = {};
calc.add = function(a, b) {
    return a + b;
}
module.exports = calc;
```

Tệp chính

- Gọi hàm add sau khi nạp tệp mô-đun
- Nạp tệp mô-đun bằng hàm require
- Đối tượng result đã nhập có thể được coi như một đối tượng module.exports

```
1   var calc2 = require('./calc2');
2   console.log(' After splitting into modules - ' +
3       'the result of calling calc2.add function : %d',
4       calc2.add(10, 10));
```



Sử dụng mô-đun bên ngoài

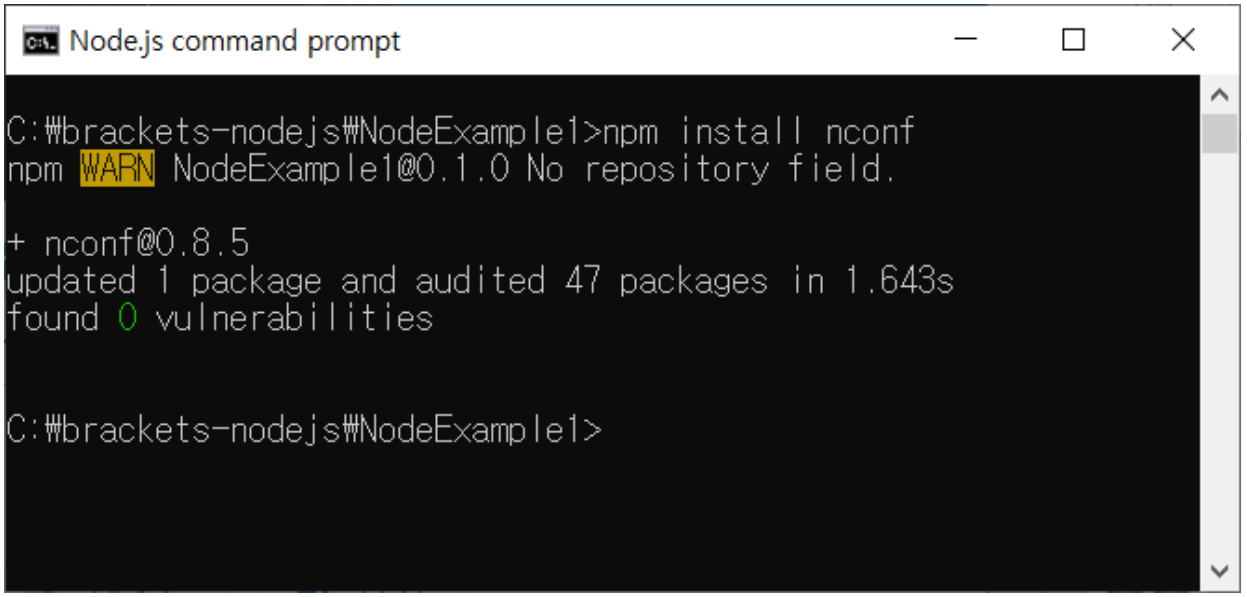
- Không sử dụng đường dẫn tương đối khi sử dụng mô-đun bên ngoài.
- mô-đun nconf cho phép truy cập vào các biến môi trường trên hệ thống

```
1   var nconf = require('nconf');
2   nconf.env();
3   console.log('value of OS environment variables : %s',
4       nconf.get('OS'));
```

Sử dụng mô-đun bên ngoài

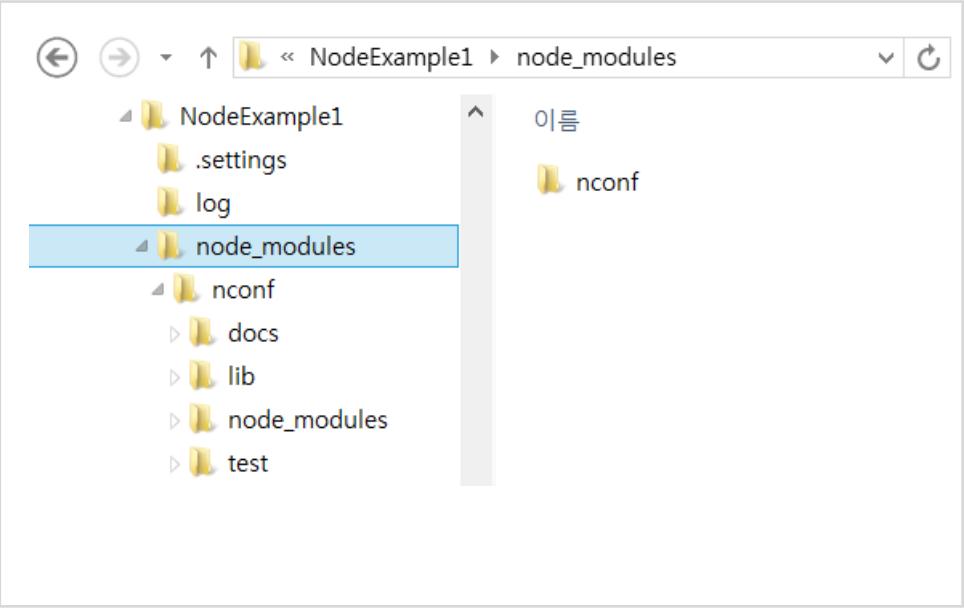
- Để sử dụng mô-đun bên ngoài, bạn cần cài đặt gói bằng npm.
- Sau khi cài đặt gói, chạy tệp JavaScript đã tạo trước đó rồi hiển thị các biến môi trường trên OS.

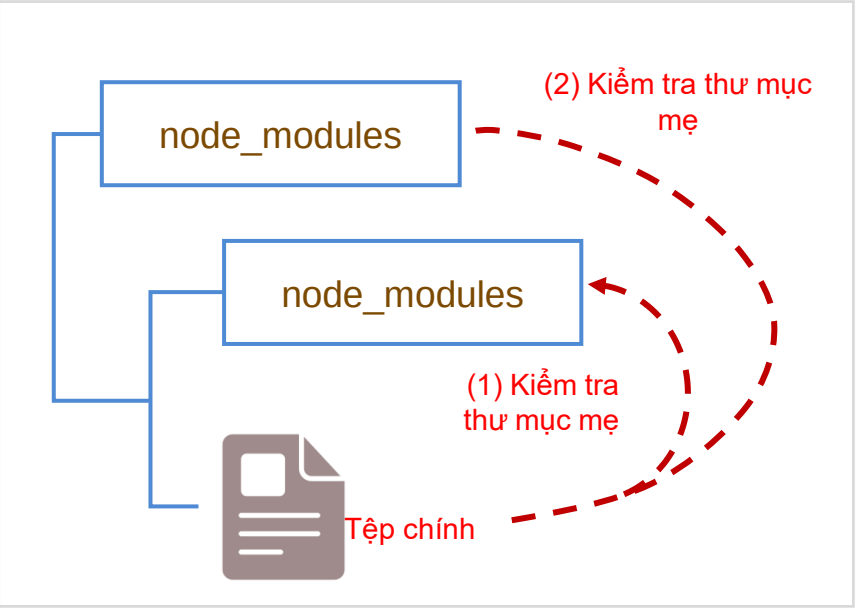
```
% npm install nconf
```



Sử dụng mô-đun bên ngoài đã cài đặt

- Các gói được cài đặt bên trong thư mục node_modules.
- Có thể được sử dụng theo từng dự án nhưng cũng có thể sử dụng trong thư mục mẹ của dự án.





tệp package.json

- Bạn có thể kiểm tra thông tin gói đã cài đặt bằng npm.

```
{
"name": "NodeExample",
"version": "0.1.0",
"description": "NodeExample1",
"main": "hello-world-server.js",
"scripts": {
"test": "echo \"Error: no test specified! Configure in package.json\" && exit 1"
},
"repository": "",
"keywords": [
"node.js",
"eclipse",
"nodeclipse"
],
"author": "",
"license": "MIT",
"readmeFilename": "README.md"
}
```

Thêm thông tin vào Package.json và xóa các gói

- Xóa bằng cách sử dụng 'npm uninstall nconf'.
- Thêm thông tin gói vào tệp 'package.json' với tùy chọn 'npm install nconf --save'.
- Nếu bạn đã sao chép thư mục dự án, hãy sử dụng lệnh npm install' để cài đặt tất cả các gói bằng thông tin
- gói có trong 'package.json'.

```
{
    ....
    ….
    "dependencies": {
        "nconf": "^0.3.4"
    }
}
```

Bài 1.

Tổng quan về Node.js

- 1.1. Tổng quan về Node.js
- 1.2. Cài đặt các công cụ phát triển
- 1.3. Tạo dự án node đầu tiên
- 1.4. Gửi log lên console
- 1.5. Sử dụng Mô-đun trong Node
- 1.6. Sử dụng mô-đun tích hợp sẵn

Mô-đun tích hợp

- Mô-đun được cung cấp khi cài đặt node.
- http://nodejs.org/api có sẵn trên trang web.
- Mô-đun trên hệ điều hành thông báo cho bạn thông tin về hệ thống.

Tên đối tượng toàn cục	Giải thích
hostname()	Cho bạn biết tên máy chủ của hệ điều hành.
totalmem()	Cho bạn biết tổng dung lượng bộ nhớ của hệ thống.
freemem()	Cho bạn biết dung lượng còn trống trên hệ thống.
cpus()	Cho bạn biết thông tin về CPU
networkInterfaces()	Trả về một đối tượng array chứa thông tin về giao diện mạng.

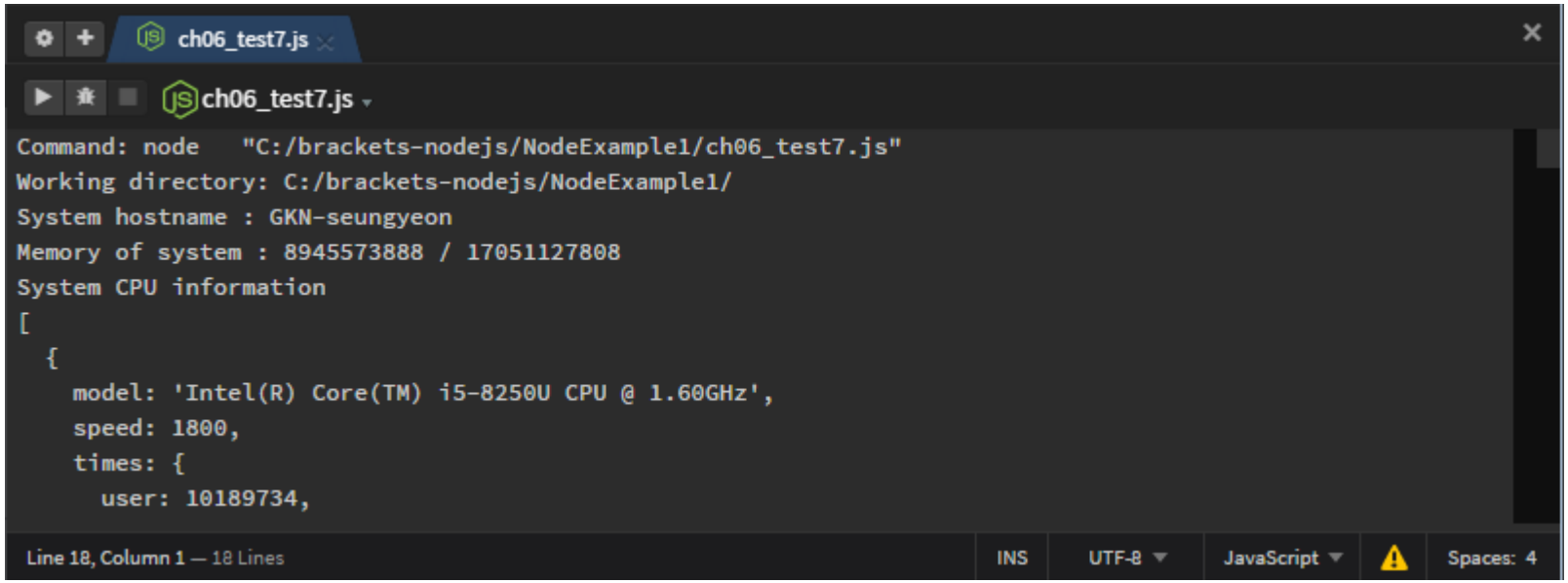
Mô-đun trên hệ điều hành

- Gọi mô-đun trên hệ điều hành bằng hàm 'require'.

```
1  var os = require('os');
2  console.log('System hostname:% s', os.hostname ());
3  console.log ('Memory of system:% d /% d',
4      os.freemem (), os.totalmem ());
5  console.log('System CPU information \ n');
6  console.dir(os.cpus());
7  console.log ('System network interface information \ n');
8  console.dir(os.networkInterfaces());
```

Mô-đun trên hệ điều hành

- Nạp mô-đun bằng hàm require



Mô-đun đường dẫn

- Cung cấp các tính năng như phân biệt tên tệp với đường dẫn tệp

Tên đối tượng toàn cục	Giải thích
join()	Kết hợp các tên với nhau thành một đường dẫn tệp duy nhất. Đối tượng này tự động điều chỉnh dấu phân cách khi hoàn thành đường dẫn tệp
dirname()	Trả về tên thư mục từ đường dẫn tệp.
basename()	Trả về tên đường dẫn tệp mà không có phần mở rộng tệp.
extname()	Trả về phần mở rộng của tệp trong đường dẫn tệp.

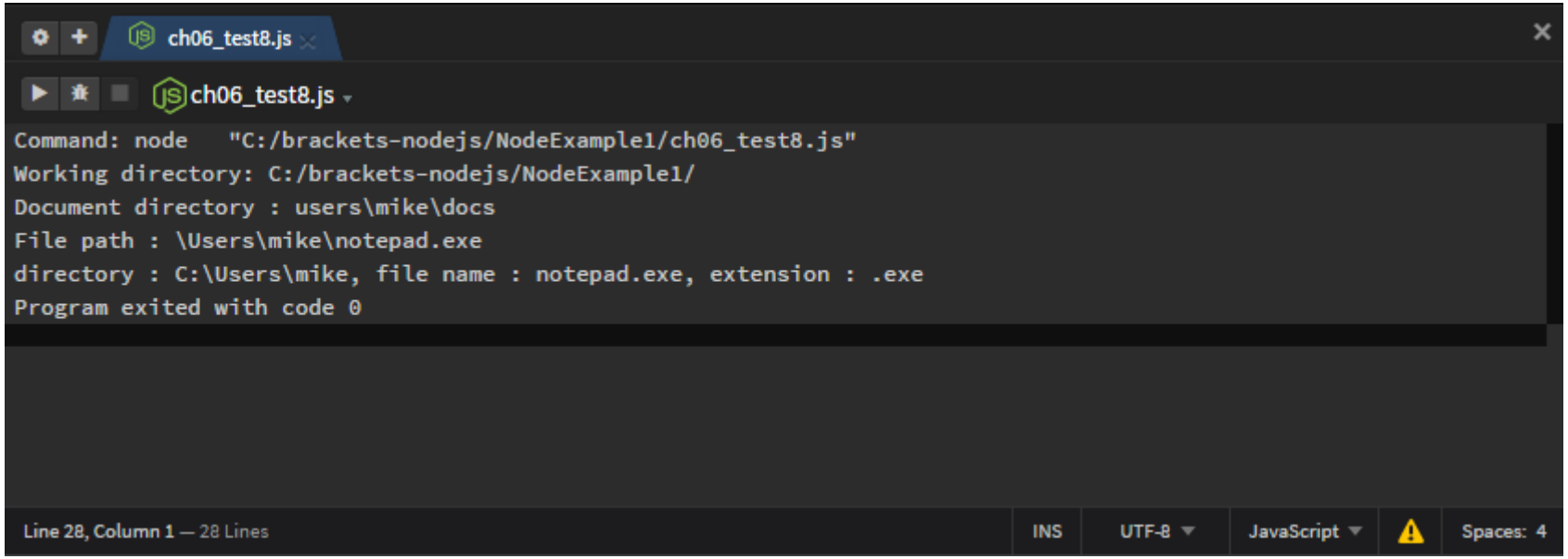
mô-đun đường dẫn

- Xác định thư mục, tên tệp và phần mở rộng trong đường dẫn tệp.

```
1   var path = require('path');
2   // Combining Directory Names
3   var directories = ["users", "mike", "docs"];
4   var docsDirectory = directories.join(path.sep);
5   console.log(' Document directory : %s', docsDirectory);
6   // Combining Directory Names and File Names
7   var curPath = path.join( separator: '/Users/mike', 'notepad.exe');
8   console.log(' File path : %s', curPath);
9
10  // Identifies directories, file names, and extensions in path
11  var filename = "C:\\Users\\mike\\notepad.exe";
12  var dirname = path.dirname(filename);
13  var basename = path.basename(filename);
14  var extname = path.extname(filename);
15  console.log('directory : %s, file name : %s, extension : %s',
16      dirname, basename, extname);
```

mô-đun đường dẫn

- Xác định thư mục, tên tệp và phần mở rộng trong đường dẫn tệp.



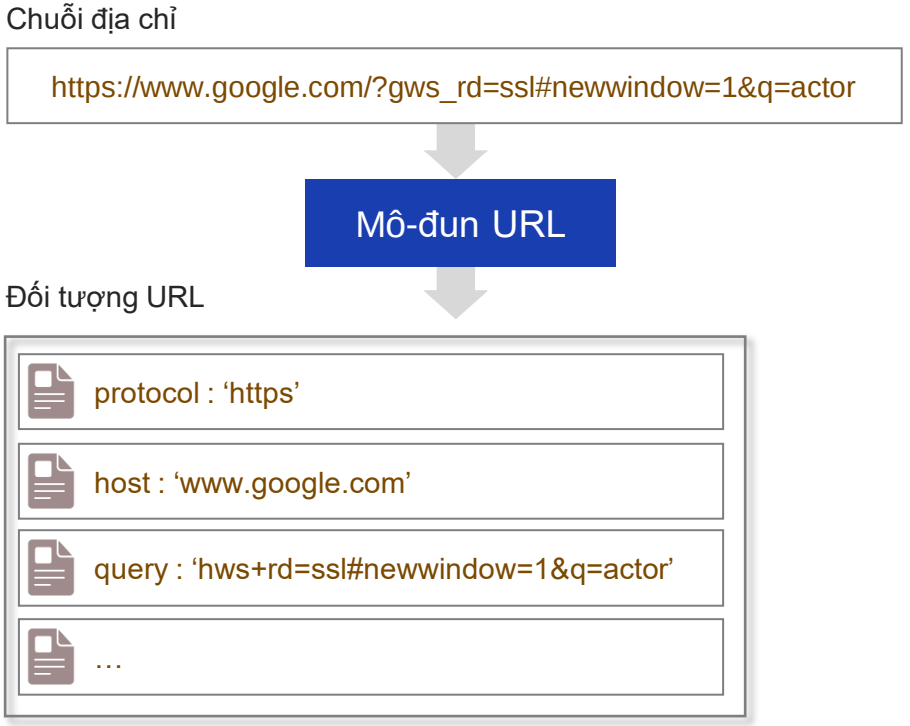
Bài 2.

Lập trình Node.js cơ bản

- 2.1. Chuỗi địa chỉ và tham số truy vấn
- 2.2. Tìm hiểu về Sự kiện
- 2.3. Sử dụng Tệp
- 2.4. Duy trì tệp Log

Mô-đun URL được sử dụng để tách các chuỗi địa chỉ

- Chuyển đổi chuỗi plain string thành đối tượng URL hoặc chuyển đổi đối tượng URL thành chuỗi plain string



Các phương thức chính của mô-đun URL

Phương thức	Giải thích
parse(urlStr, [parseQueryString = false, slashesDenoteHost = false])	Phân tích cú pháp chuỗi địa chỉ để tạo đối tượng URL
format(urlObj)	Chuyển đổi đối tượng URL thành chuỗi địa chỉ

```
1  var url = require('url');
2  var curURL = url.parse(
3      s: 'https://www.google.com/search?query=steve+jobs');
4      var curStr = url.format(curURL);
5
6  console.log('address string : %s', curStr);
7  console.dir(curURL);
```

Phân tích URL bằng phương thức parse (phân tích cú pháp)

- Màn hình hiển thị kết quả đã phân tích



Mô-đun querystring dùng để Kiểm tra tham số truy vấn

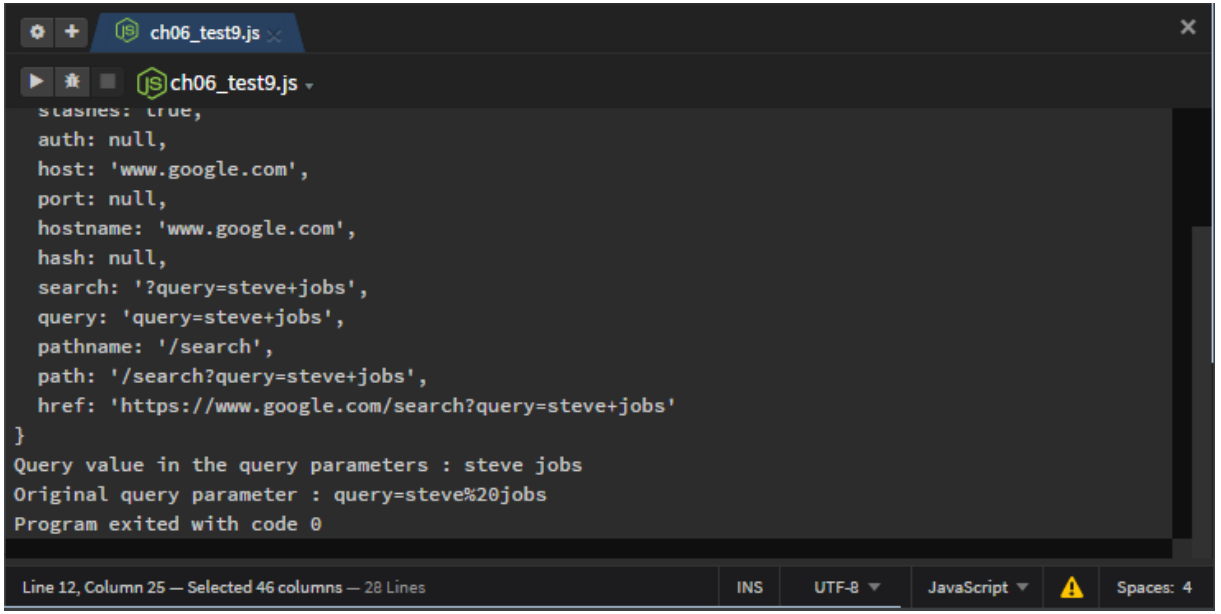
- Được sử dụng để phân tách tham số truy vấn được phân đoạn bằng ký hiệu &
- Sử dụng Phương thức parse và stringify sau khi nạp mô-đun với phương thức require

```
1   var url = require('url');
2   var curURL = url.parse(
3       's: 'https://www.google.com/search?query=steve+jobs');
4       var curStr = url.format(curURL);
5
6   console.log('address string : %s', curStr);
7   console.dir(curURL);
8
9   var querystring = require('querystring');
10  var param = querystring.parse(curURL.query);
11  console.log('query value in the query parameters : %s',
12      param.query);
13  console.log('original query parameter : %s',
14      querystring.stringify(param));
```

Các phương thức trong mô-đun querystring

- Các phương thức trong mô-đun querystring

Phương thức	Giải thích
parse()	Phân tích chuỗi tham số truy vấn để tạo đối tượng tham số truy vấn
stringfy()	Chuyển đổi đối tượng tham số truy vấn thành chuỗi



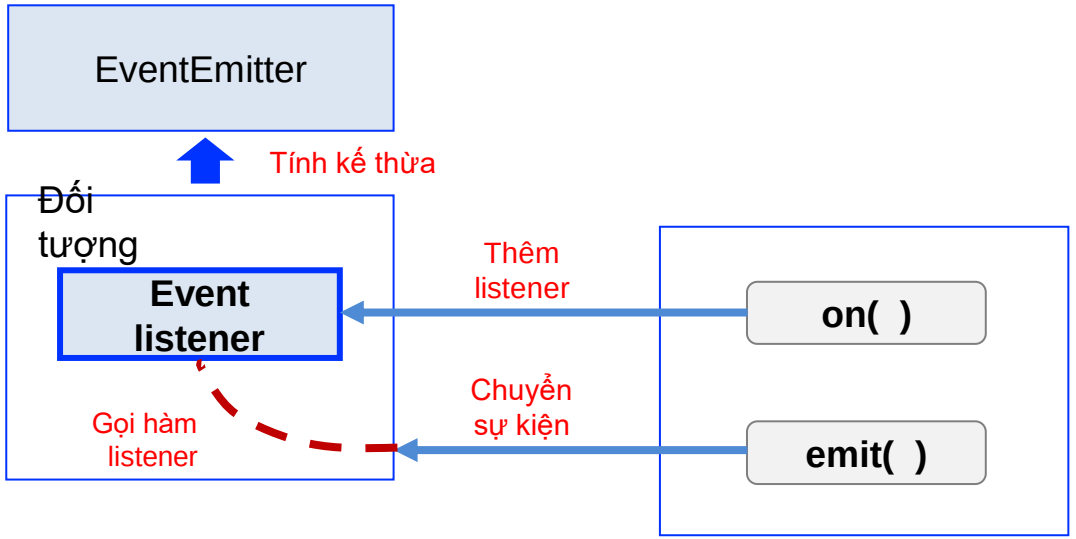
Bài 2.

Lập trình Node.js cơ bản

- 2.1. Chuỗi địa chỉ và tham số truy vấn
- 2.2. Tìm hiểu về Sự kiện
- 2.3. Sử dụng Tệp
- 2.4. Duy trì tệp Log

Sự kiện là gì?

- Chuyển dữ liệu từ bên này sang bên khác để xử lý không đồng bộ
- Sử dụng hàm EventEmitter
- Khi gửi sự kiện từ bên này sang bên khác, sử dụng phương thức emit(). Phương thức on() bổ sung event
- listener cho đối tượng mà sự kiện được chuyển đến



Gửi và nhận sự kiện

- Đăng ký listener với on() và gửi sự kiện bằng emit()

Phương thức	Giải thích
open(path, flags, [mode], [callback])	Mở tệp tin
read(fd, buffer, offset, length, position, [callback])	Đọc nội dung tệp của phần được chỉ định
write(fd, buffer, offset, length, position, [callback])	Ghi dữ liệu vào phần được chỉ định của tệp
close(fd, [callback])	Đóng tệp

```
1  process.on('tick', function(count) {
2      console.log('tick event occurs: %s', count);
3  });
4  setTimeout( handler: function() {
5      console.log('attempt to pass tick event in 2 seconds.');
6      process.emit('tick', '2');
7  }, timeout: 2000);
```

Cấu hình đối tượng calculator thành mô-đun

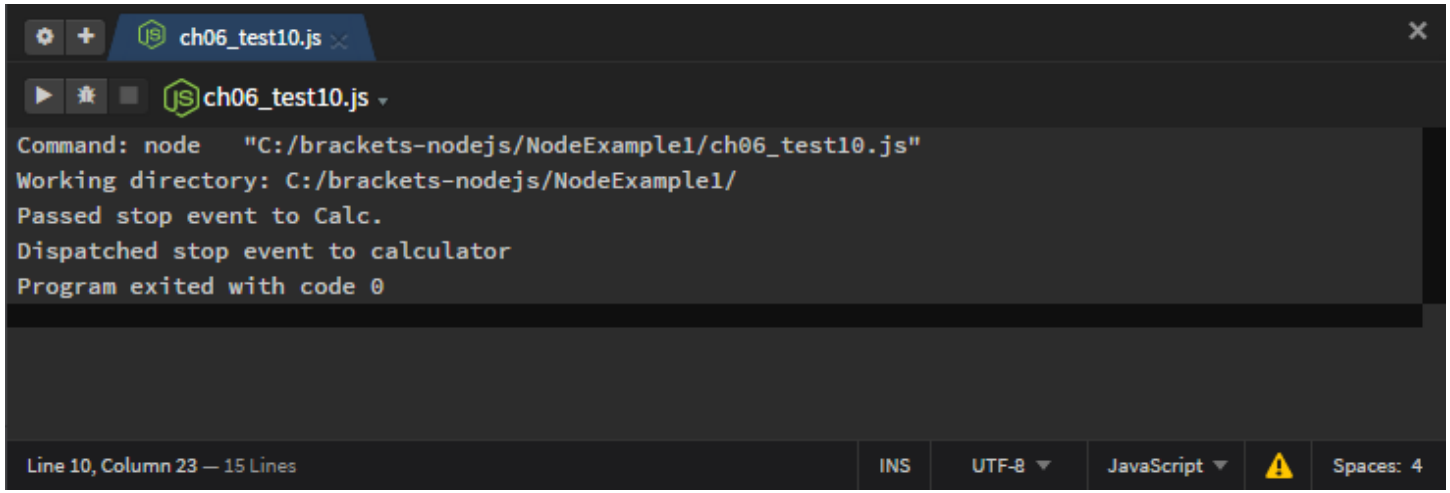
- Các phương thức emit và on đều khả dụng nếu đối tượng calculator kế thừa EventEmitter

```
1  var util = require('util');
2  var EventEmitter = require('events').EventEmitter;
3  var Calc = function() {
4      var self = this;
5
6      this.on('stop', function() {
7          console.log('Passed stop event to Calc.');
8      });
9  };
10 util.inherits(Calc, EventEmitter);
11 Calc.prototype.add = function(a, b) {
12     return a + b;
13 }
14 module.exports = Calc;
15 module.exports.title = 'calculator';
```

Sử dụng Đối tượng Calculator trong Tệp tin chính

- Gửi sự kiện đến emit

```
1    var Calc = require('./calc3');
2    var calc = new Calc();
3    calc.emit('stop');
4    console.log('stop event sent to ' + Calc.title);
```



Bài 2.

Sử dụng Tệp

- 2.1. Chuỗi địa chỉ và tham số truy vấn
- 2.2. Tìm hiểu về Sự kiện
- 2.3. Sử dụng Tệp
- 2.4. Duy trì tệp Log

Hệ thống tệp của node

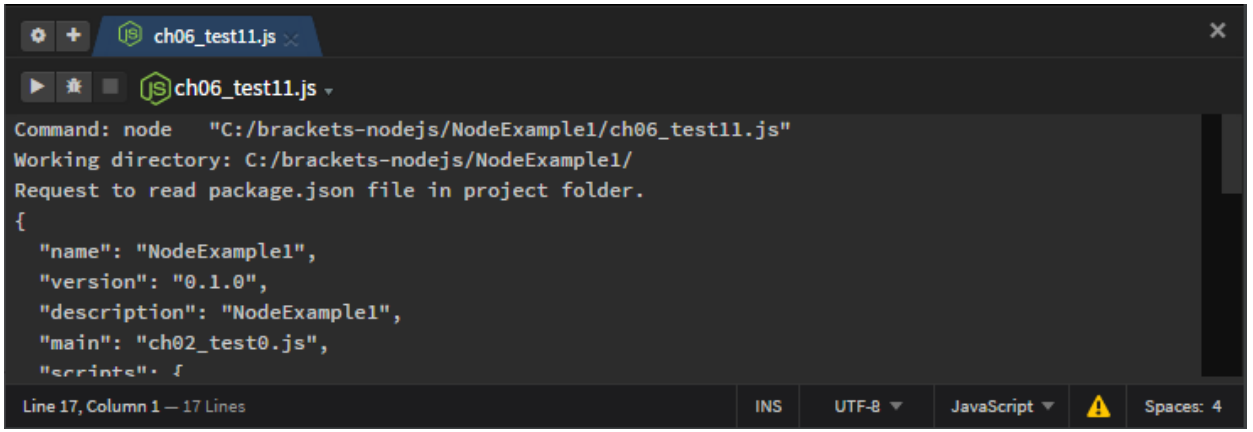
- Cung cấp I/O đồng bộ và không đồng bộ
- Lưu ý rằng I/O đồng bộ sẽ chờ cho đến khi hoàn thành thao tác tệp
- Trong phương thức I/O đồng bộ, từ Sync theo sau

```
1  var fs = require('fs');
2
3  var data = fs.readFileSync('./package.json', 'utf8');
4  console.log(data);
```

Đọc tệp không đồng bộ

- Chuyển hàm Callback làm tham số, sử dụng phương thức readFile

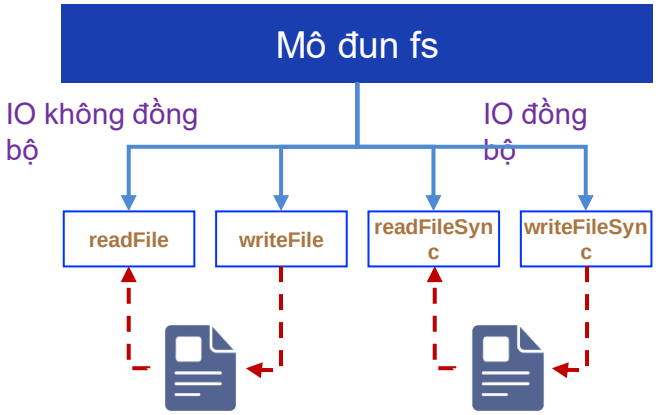
```
1  var fs = require('fs');
2
3  fs.readFile('./package.json', 'utf8', function(err, data) {
4      console.log(data);
5  });
6
7  console.log(' Request to read package.json file ' +
8      'in the project folder.');
```



Phương thức chính của mô-đun fs

- Đọc bằng readFile và ghi bằng writeFile

Phương thức	Giải thích
readFile(filename, [encoding], [callback])	Đọc tệp bằng I/O không đồng bộ
readFileSync(filename, [encoding])	Đọc tệp bằng I/O đồng bộ
writeFile(filename, data, encoding='utf8', [callback])	Ghi tệp vào I/O không đồng bộ
writeFileSync(filename, data, encoding='utf8')	Ghi tệp vào I/O đồng bộ
stat(path, callback)	Kiểm tra thông tin trong tệp



Ghi tệp không đồng bộ

- Truyền hàm callback cho tham số bằng phương thức readFile

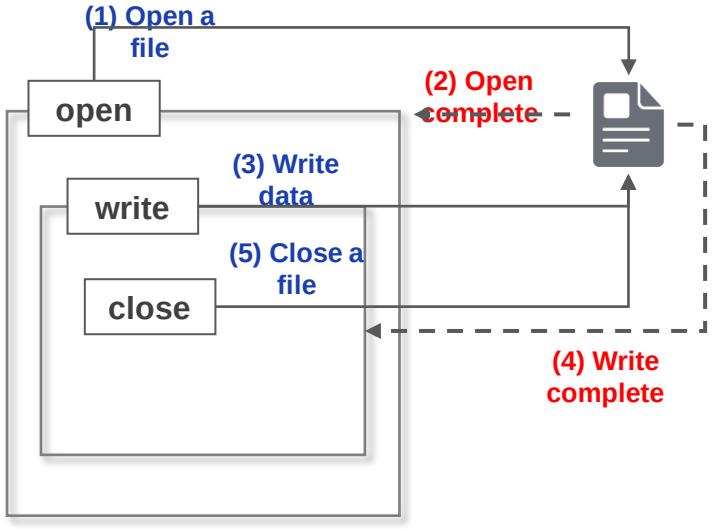
```
var fs = require('fs');

fs.writeFile('./output.txt', 'Hello World!', function(err) {
    if(err) {
        console.log('Error : ' + err);
    }
    console.log('Complete writing data to output.txt file');
});
```

Đọc hoặc ghi trong khi mở và đóng tệp

- sử dụng các phương thức open, read, write, close, v.v.

Phương thức	Giải thích
open(path, flags, [mode], [callback])	Mở tệp tin
read(fd, buffer, offset, length, position, [callback])	Đọc nội dung tệp của phần được chỉ định
write(fd, buffer, offset, length, position, [callback])	Ghi dữ liệu vào phần được chỉ định của tệp
close(fd, [callback])	Đóng tệp



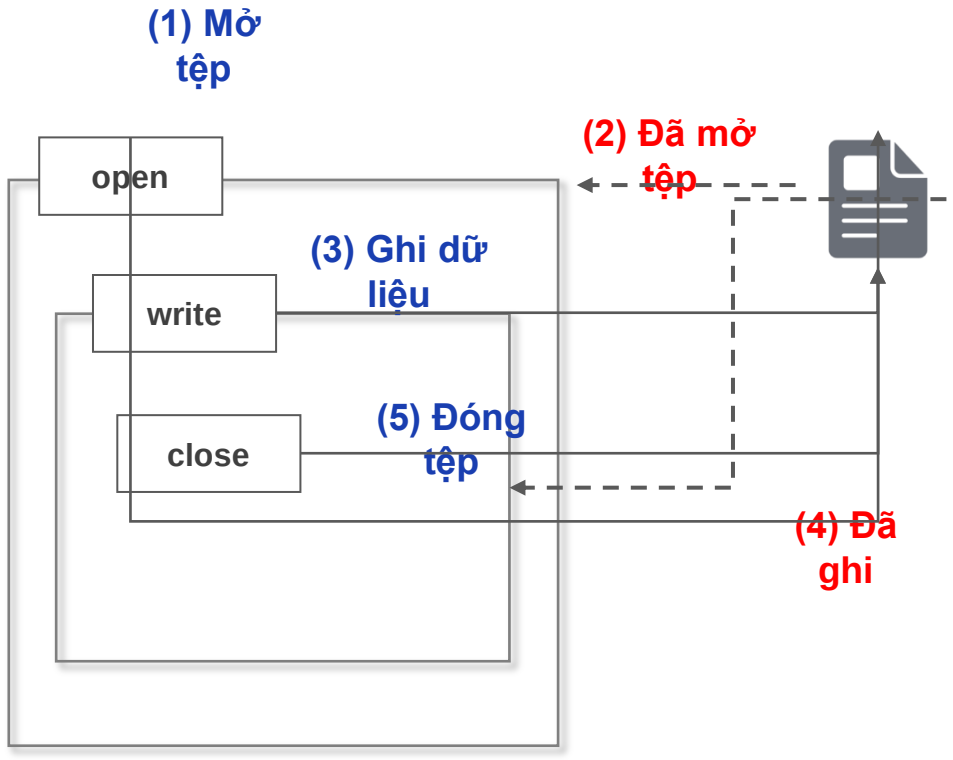
Mở tệp trực tiếp và ghi dữ liệu

- Mở và viết.

```
1   var fs = require('fs');
2
3   fs.open('./output.txt', 'w', function(err, fd) {
4       if(err) throw err;
5       var buf = new Buffer('Hello!\n');
6       fs.write(fd, buf, 0, buf.length, null,
7           function(err, written, buffer) {
8           if(err) throw err;
9           console.log(err, written, buffer);
10
11          fs.close(fd, function() {
12              console.log('Open file, write data and close file.');
13          });
14      });
15  });
```

Luồng xử lý tệp

- Tiến trình mở bằng phương thức open và ghi bằng phương thức write



Tự mở tệp và đọc

- Mở, đọc và sử dụng bộ đệm

```
var fs = require('fs');

fs.open('./output.txt', 'r', function(err, fd) {
    if(err) throw err;
    var buf = new Buffer(10);
    console.log(' Buffer type : %s', Buffer.isBuffer(buf));
    fs.read(fd, buf, 0, buf.length, null,
        function(err, bytesRead, buffer) {
        if(err) throw err;
        var inStr = buffer.toString('utf8', 0, bytesRead);
        console.log(' Data read from file : %s', inStr);
        console.log(err, bytesRead, buffer);
        fs.close(fd, function() {
            console.log('Open the output.txt file, ' +
                'read it, complete.');
        });
    });
});
```

Kiểm tra thông tin

- Kiểm tra thông tin tệp bằng stat
- Cung cấp các hàm như isFile(), isDirectory() và isFIFO()

```
1  var fs = require('fs');
2
3  fs.stat('./stat.js', function(err, stats){
4      if (err) throw err;
5
6      console.log(stats);
7      console.log('isFile() : %s', stats.isFile());
8  });
```

- Màn hình kết quả.

```
{ dev: -55011900,
  mode: 33206,
  nlink: 1,
  uid: 0,
  gid: 0,
  rdev: 0,
  blksize: undefined,
  ino: 56013520365545000,
  size: 168,
  blocks: undefined,
  atime: Sun May 29 2019 20:50:49 GMT+0900 (Korea Standard Time),
  mtime: Sun May 29 2019 20:50:49 GMT+0900 (Korea Standard Time),
  ctime: Sun May 29 2019 20:50:49 GMT+0900 (Korea Standard Time),
  birthtime: Sun May 29 2019 20:49:59 GMT+0900 (Korea Standard Time) }
isFile() : true
```

Sử dụng bộ đệm

- Tạo bộ đệm mới bằng phương thức 'new' và sử dụng các phương thức như 'Buffer.isBuffer ()' và 'Buffer.concat ()'.

```
var output = ' Hi 1!';
var buffer1 = new Buffer(10);
var len = buffer1.write(output, 'utf8');
console.log('The string in the first buffer : %s',
    buffer1.toString());

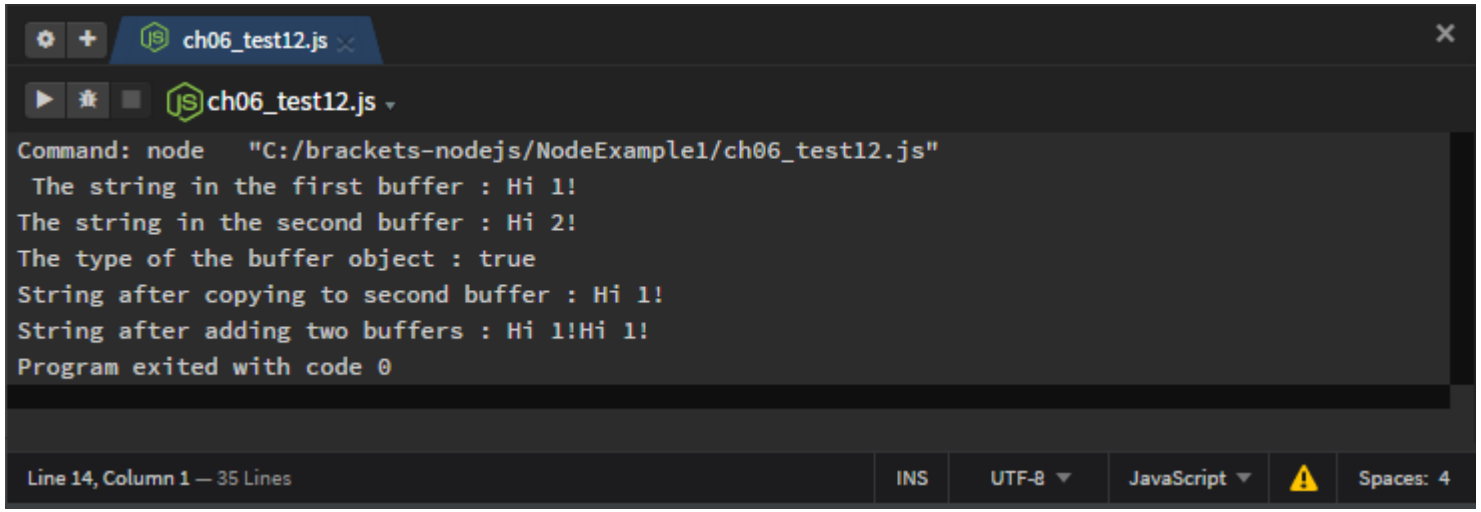
var buffer2 = new Buffer(' Hi 2!', 'utf8');
console.log('The string in the second buffer : %s',
    buffer2.toString());

console.log('The type of the buffer object : %s',
    Buffer.isBuffer(buffer1));

var byteLen = Buffer.byteLength(output);
var str1 = buffer1.toString('utf8', 0, byteLen);
var str2 = buffer2.toString('utf8');

buffer1.copy(buffer2, 0, 0, len);
console.log('The string after copying to second buffer : %s',
    buffer2.toString('utf8'));
```

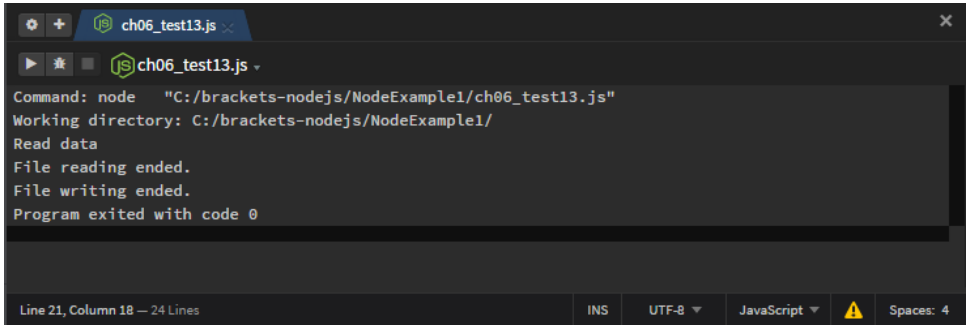
- Tạo bộ đệm mới bằng phương thức 'new' và sử dụng các phương thức như 'Buffer.isBuffer () và Buffer.concat ()'.



Đọc các tệp và ghi vào luồng

- Mở để đọc bằng 'createReadStream', Mở để ghi bằng 'createWriteStream'

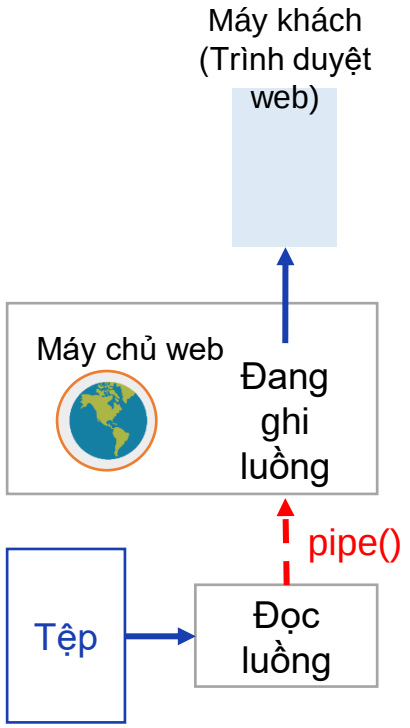
```
1   var fs = require('fs');
2
3   var infile = fs.createReadStream(
4       './output.txt', {flags: 'r'} );
5   var outfile = fs.createWriteStream(
6       './output2.txt', {flags: 'w'});
7
8   infile.on('data', function(data) {
9       console.log(' Read data ', data);
10      outfile.write(data);
11  });
12
13  infile.on('end', function() {
14      console.log('File read end.');
15      outfile.end(function() {
16          console.log(' File read end.');
17      })
18  });
```



Đọc và phản hồi với nội dung tệp được yêu cầu từ mô-đun “http”

- Đọc tệp theo luồng và kết nối dữ liệu với một 'pipe' (đường ống).

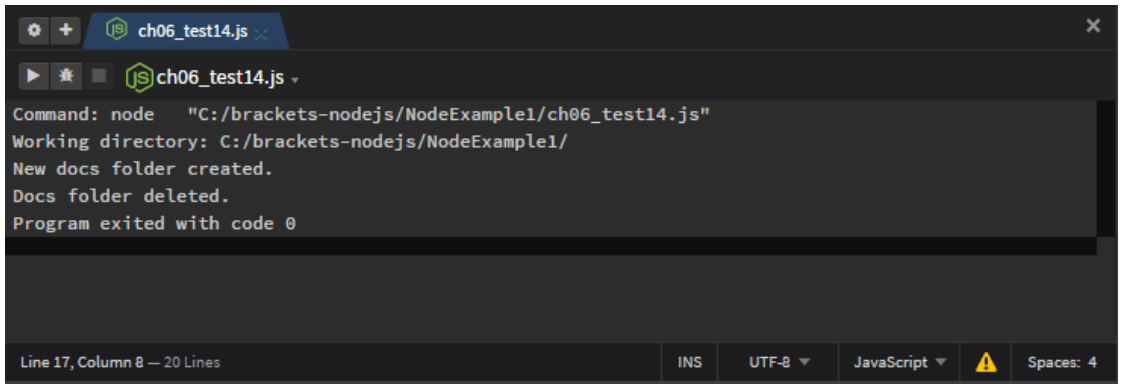
```
1   var fs = require('fs');
2   var http = require('http');
3
4   var server = http.createServer(function(req, res) {
5       // Read the file and associate it with the
6       // response stream as 'pipe ()'.
7       var instream = fs.createReadStream('./output.txt');
8       instream.pipe(res);
9   });
10
11  server.listen(7001, '127.0.0.1');
```



Tạo và xóa thư mục mới

- Tạo thư mục bằng phương thức 'mkdir' và xóa thư mục bằng phương thức “rmdir”.

```
1   var fs = require('fs');
2   fs.mkdir('./docs', '0666', function(err) {
3       if(err) throw err;
4       console.log(' I have created a new docs folder.');
5
6       fs.rmdir('./docs', function(err) {
7           if(err) throw err;
8           console.log('docs folder deleted.');
9       });
10  });
```



Bài 2.

Duy trì tệp Log

- 2.1. Chuỗi địa chỉ và tham số truy vấn
- 2.2. Tìm hiểu về Sự kiện
- 2.3. Sử dụng Tệp
- 2.4. Duy trì tệp Log

Ghi log bằng mô-đun “winston”

- Một vài mô-đun có thể giúp bạn ghi log.
- Bạn nên ghi log theo ngày.
- Có nhiều cài đặt khác có thể thực thi được trong mô-đun 'winston'.

```
1  var winston = require('winston');    // Log processing module
2  var winstonDaily = require('winston-daily-rotate-file');
3  // Daily log processing module
4  var moment = require('moment');    // Time processing module
5
6  function timeStampFormat() {
7      return moment().format('YYYY-MM-DD HH:mm:ss.SSS ZZ');
8      // '2019-05-01 20:14:28.500 +0900'
9  };
```

cấu hình mô-đun 'winston'

- Bạn có thể thực hiện các cài đặt khác nhau, bao gồm cả vị trí các tệp log không thay đổi.

```
var logger = new (winston.Logger)({
    transports: [
        new (winstonDaily)({
            name: 'info-file',
            filename: './log/server',
            datePattern: '_yyyy-MM-dd.log',
            colorize: false,
            maxsize: 50000000,
            maxFiles: 1000,
            level: 'info',
            showLevel: true,
            json: false,
            timestamp: timeStampFormat
        }),
        new (winston.transports.Console)({
            name: 'debug-console',
            colorize: true,
            level: 'debug',
            showLevel: true,
            json: false,
            timestamp: timeStampFormat
        })
    ],
```

```
    exceptionHandlers: [
        new (winstonDaily)({
            name: 'exception-file',
            filename: './log/exception',
            datePattern: '_yyyy-MM-dd.log',
            colorize: false,
            maxsize: 50000000,
            maxFiles: 1000,
            level: 'error',
            showLevel: true,
            json: false,
            timestamp: timeStampFormat
        }),
        new (winston.transports.Console)({
            name: 'exception-console',
            colorize: true,
            level: 'debug',
            showLevel: true,
            json: false,
            timestamp: timeStampFormat
        })
    ]
});
```

Sử dụng mô-đun winston

- Trong một đoạn mã sao chép tệp, ghi log bằng phương thức 'logger.info ()'.

```
var fs = require('fs');

var inname = './output.txt';
var outname = './output2.txt';

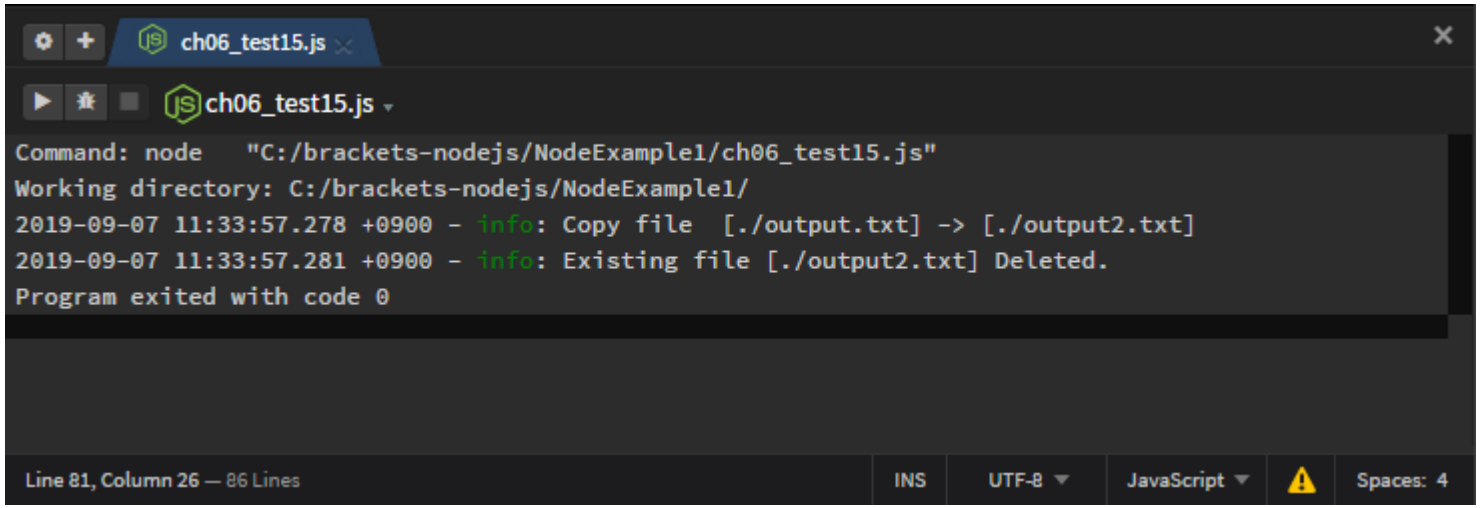
fs.exists(outname, function (exists) {
    if (exists) {
        fs.unlink(outname, function (err) {
            if (err) throw err;
            logger.info(' Existing file [' + outname +'] Deleted.');
        });
    }

    var infile = fs.createReadStream(inname, {flags: 'r'} );
    var outfile = fs.createWriteStream(outname, {flags: 'w'});

    infile.pipe(outfile);
    logger.info(' Copy file [' + inname + '] -> [' + outname + ']');
});
```

- Cài đặt mô-đun bên ngoài

```
% npm install winston –save
% npm install winston-daily-rotate-file –save
% npm install moment --save
```



Bài 3.

Raspberry Pi với Node-RED

- 3.1. Cài đặt ban đầu
- 3.2. Điều khiển đèn LED bằng Node-RED
- 3.3. Phát hiện truy cập bằng Node-RED

Install node.js on Raspberry Pi

Cài đặt NodeJS mới nhất

- Cài đặt 'node.js' trên 'Raspberry Pi'

```
$ wget -O - https://raw.githubusercontent.com/meech-ward/NodeJs-Raspberry-Pi/master/Install-Node.sh | sudo bash;
```

- Kiểm tra phiên bản Node.js trên Raspberry Pi.

```
$ node –v
```

```
pi@raspberrypi:~ $ node -v
v16.10.0
```

- Bạn có thể sử dụng 1 bài thử nghiệm với câu lệnh đơn giản nếu cài đặt thành công.

```
pi@raspberrypi:~ $ node
Welcome to Node.js v16.10.0.
Type ".help" for more information.
> var greeting = "Hello"
undefined
> console.log(greeting)
Hello
undefined
>
pi@raspberrypi:~ $
```

Cài đặt Node-RED

- Trước khi bắt đầu Cài đặt, chúng ta nên cập nhật các thư viện hiện có.
  - Luôn cập nhật thư viện bằng cách nhập liên tiếp các lệnh sau:

```
$ sudo apt update
$ sudo apt upgrade
```

  - Tải xuống Node-RED. (mất tương đối thời gian.)
  - Nhập y cho hai câu hỏi ở giữa.

```
$ bash <(curl -sL https://raw.githubusercontent.com/node-red/linux-installers/master/deb/update-nodejs-and-nodered)
```

- Quy trình cài đặt Node-RED

```
Running Node-RED install for user pi at /home/pi on raspbian


This can take 20-30 minutes on the slower Pi versions - please wait.

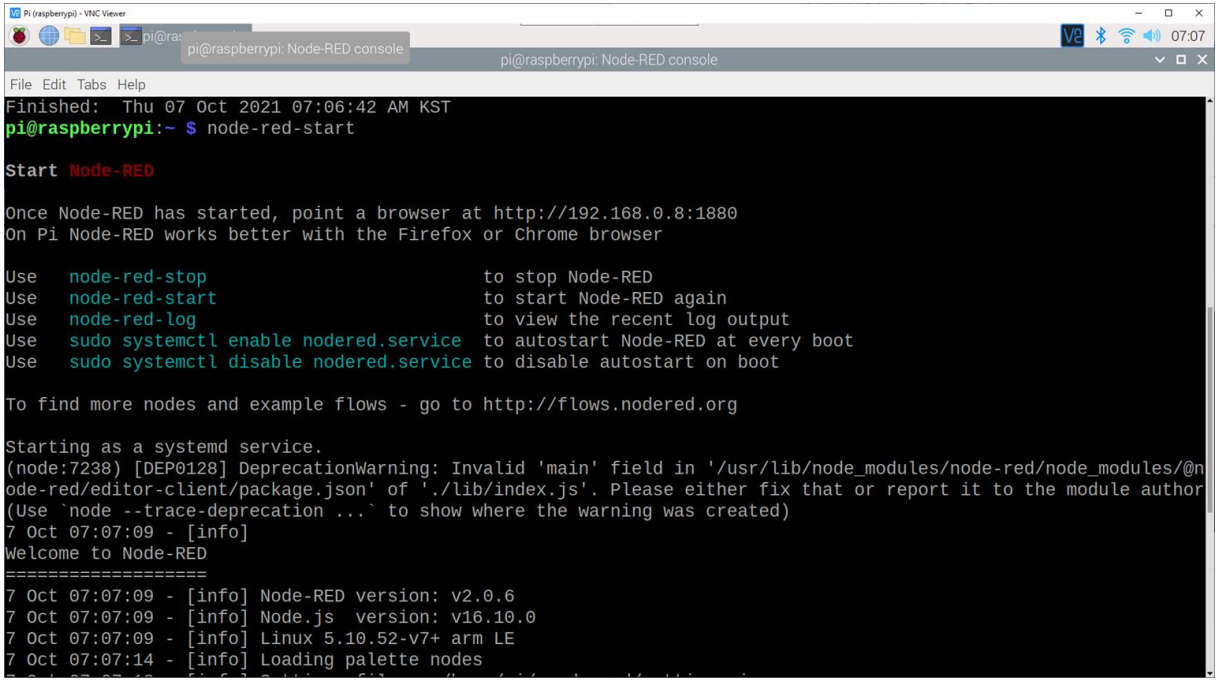
  Stop Node-RED                       ✔
  Remove old version of Node-RED      ✔
  Node option not specified           :   --node12 or --node14
  Leave existing Node.js              :   v16.10.0   Npm 7.24.0
  Clean npm cache                     -
  Install Node-RED core               ✔   2.0.6
  Move global nodes to local
  Npm rebuild existing nodes
  Install extra Pi nodes
  Add shortcut commands
  Update systemd script


Any errors will be logged to   /var/log/nodered-install.log
```

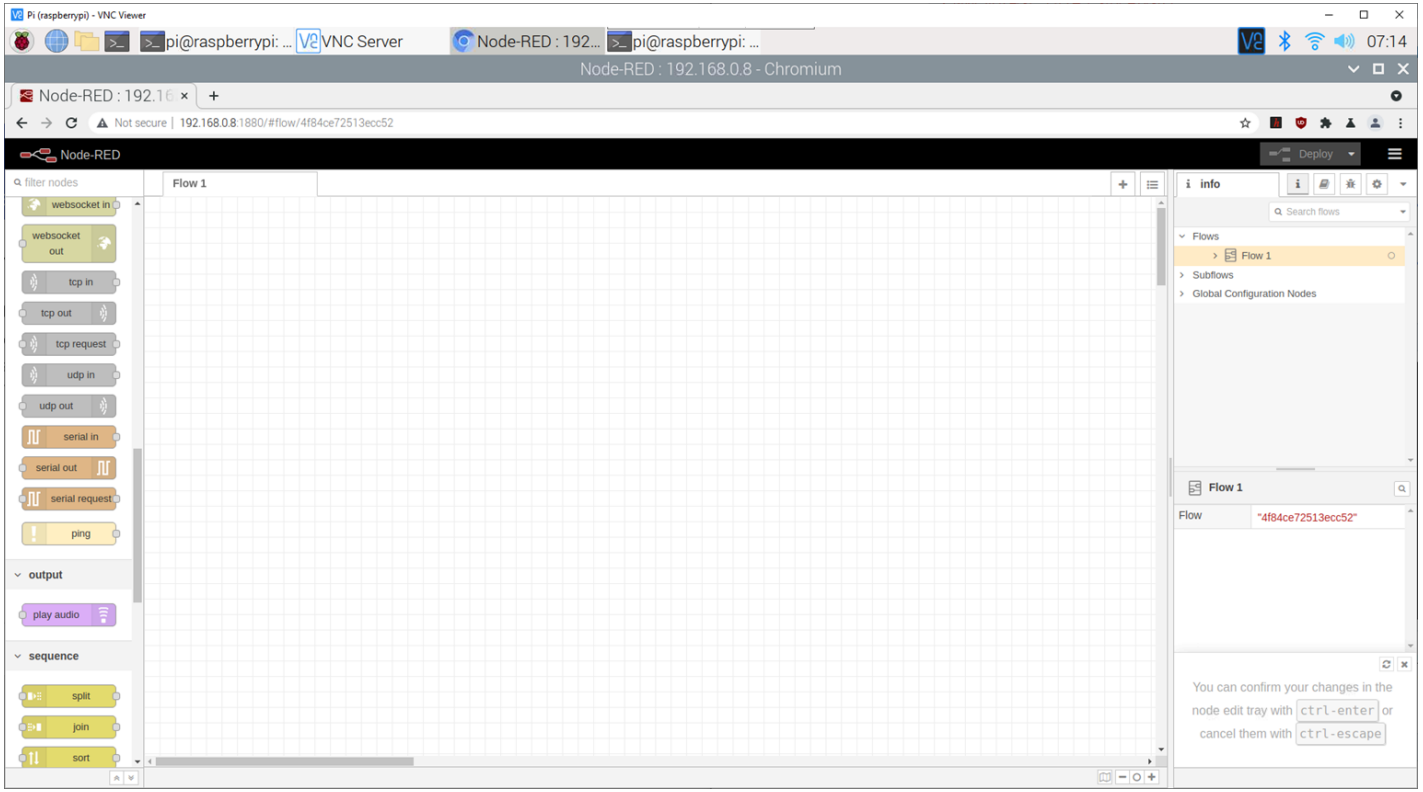
Khởi động Node-RED

- Khởi động Node-RED
  - Thực thi với Node JS thông qua lệnh thực thi sau:

```
$ node-red-start
```



- Khởi động Node-RED
  - Truy cập trang web http://{Raspberry Pi's IP Address}:1880 trên trình duyệt của bạn.
  - Trình duyệt NC và trình duyệt Windows đều có sẵn, nhưng ở đây, hãy sử dụng trình duyệt trong chính Raspberry Pi để truy cập.



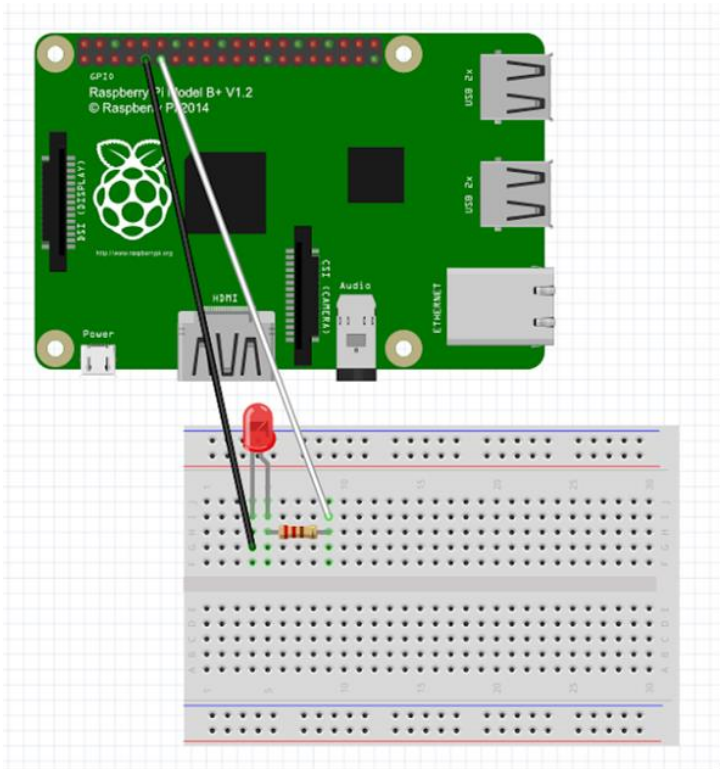
Bài 3.

Raspberry Pi với Node-RED

- 3.1. Cài đặt ban đầu
- 3.2. Điều khiển đèn LED bằng Node-RED
- 3.3. Phát hiện truy cập bằng Node-RED

Điều khiển đèn LED bằng Node-RED

- Ví dụ điều khiển LED đơn giản cho việc sử dụng Node-RED

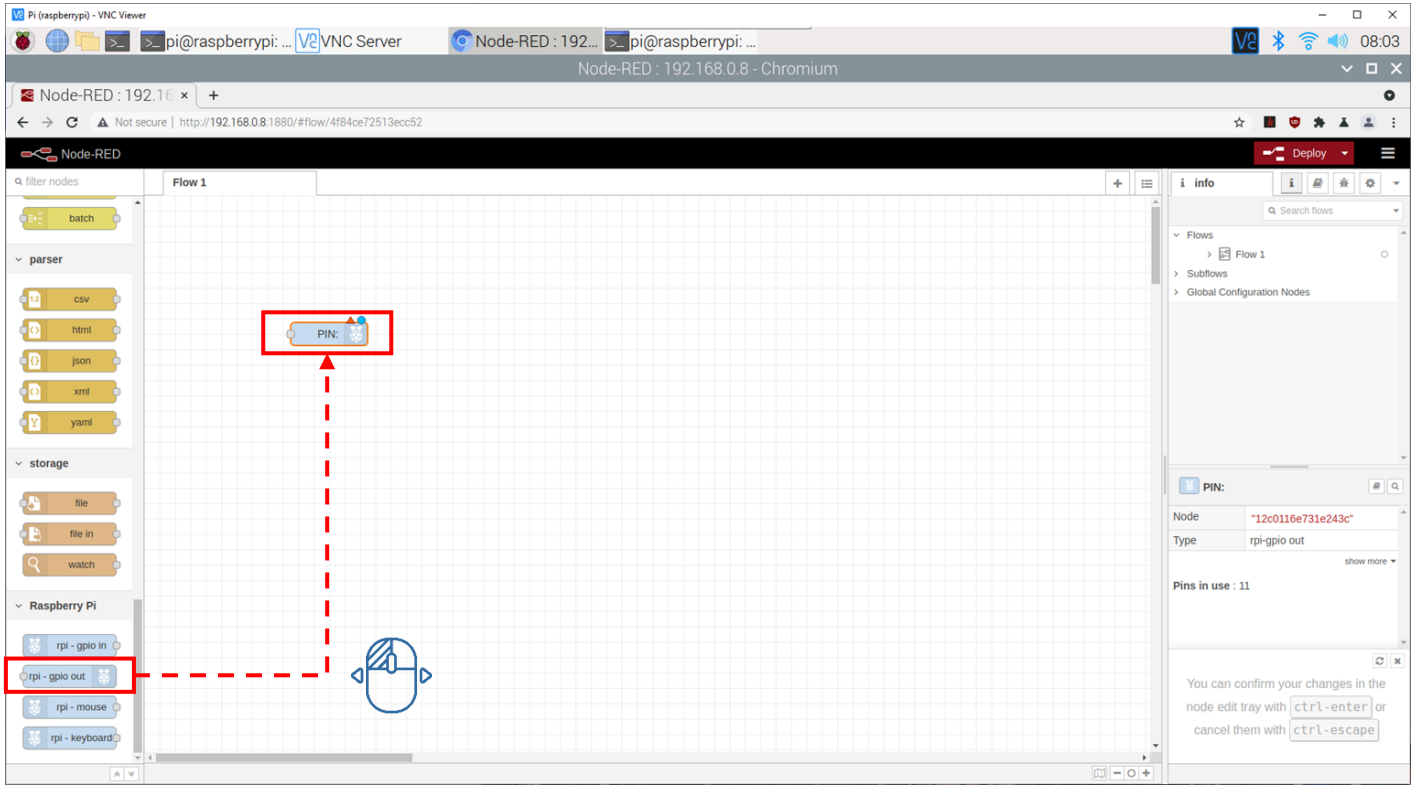


- Một ví dụ về điều khiển đèn LED bằng cách gửi tín hiệu đến GPIO bằng Node-RED.

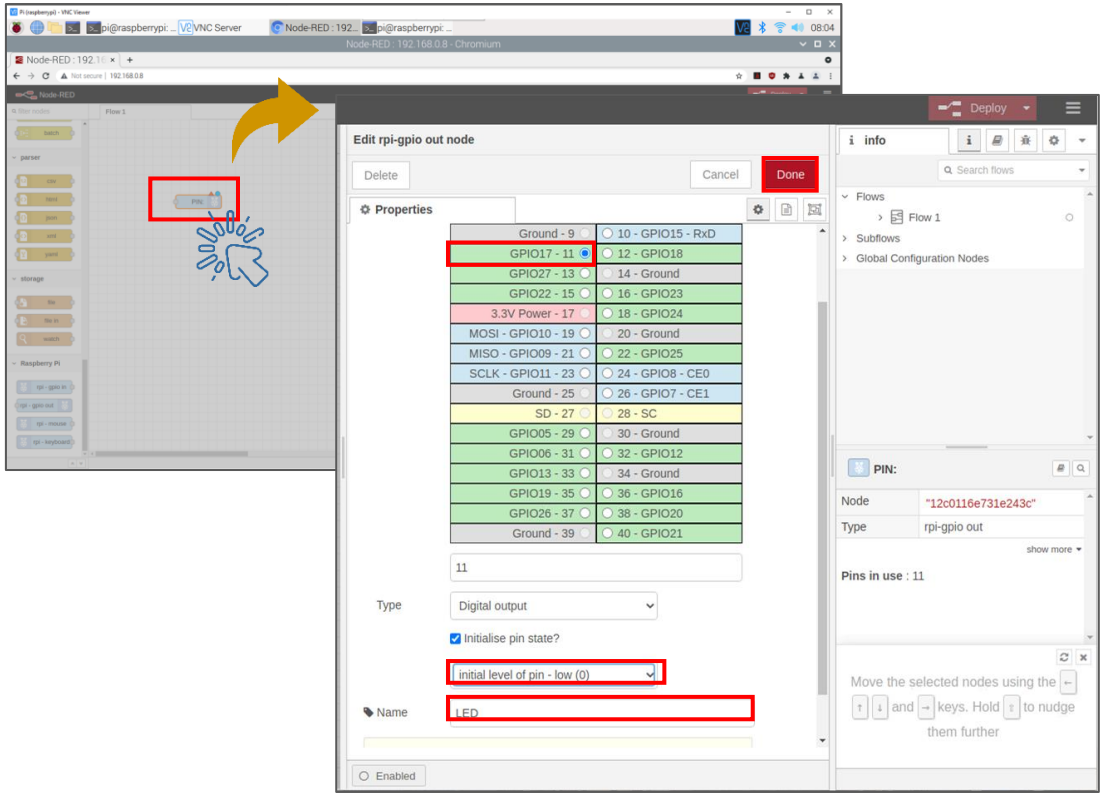
- Sơ đồ mạch
  - Raspberry Pi GPIO 17 – Resistor – LED Anode(+)
  - LED Cathode(-) - GND

GPIO chân

- Thêm chân vào Node-RED
  - Tìm rpi-gpio-out trong danh mục Raspberry Pi và kéo nó vào lưới để thêm chân.

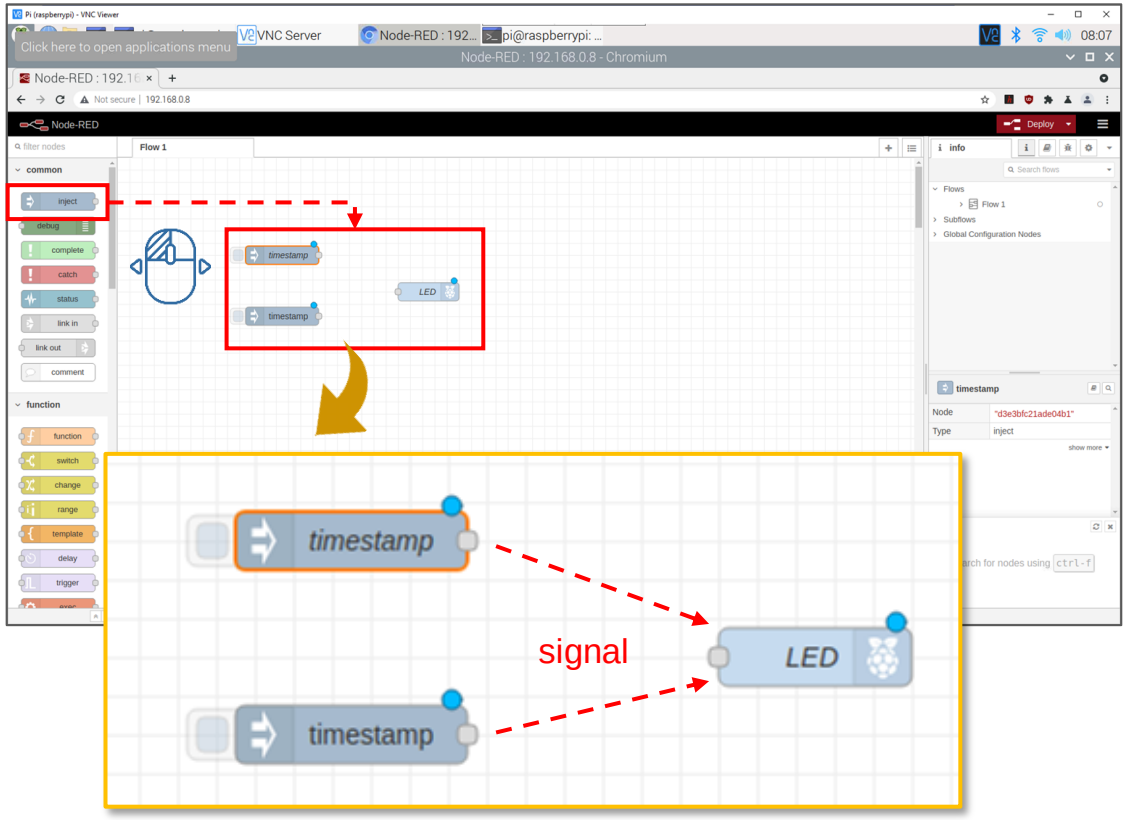


- Chỉnh sửa thuộc tính chân
  - Nhấp đúp vào chân đã tạo để chỉnh sửa Thuộc tính.



- Chọn GPIO 17, đặt Trạng thái khởi tạo chân thành trạng thái thấp và thay đổi Tên thành LED.
- Sau khi chỉnh sửa, nhấp vào nút Done màu đỏ.

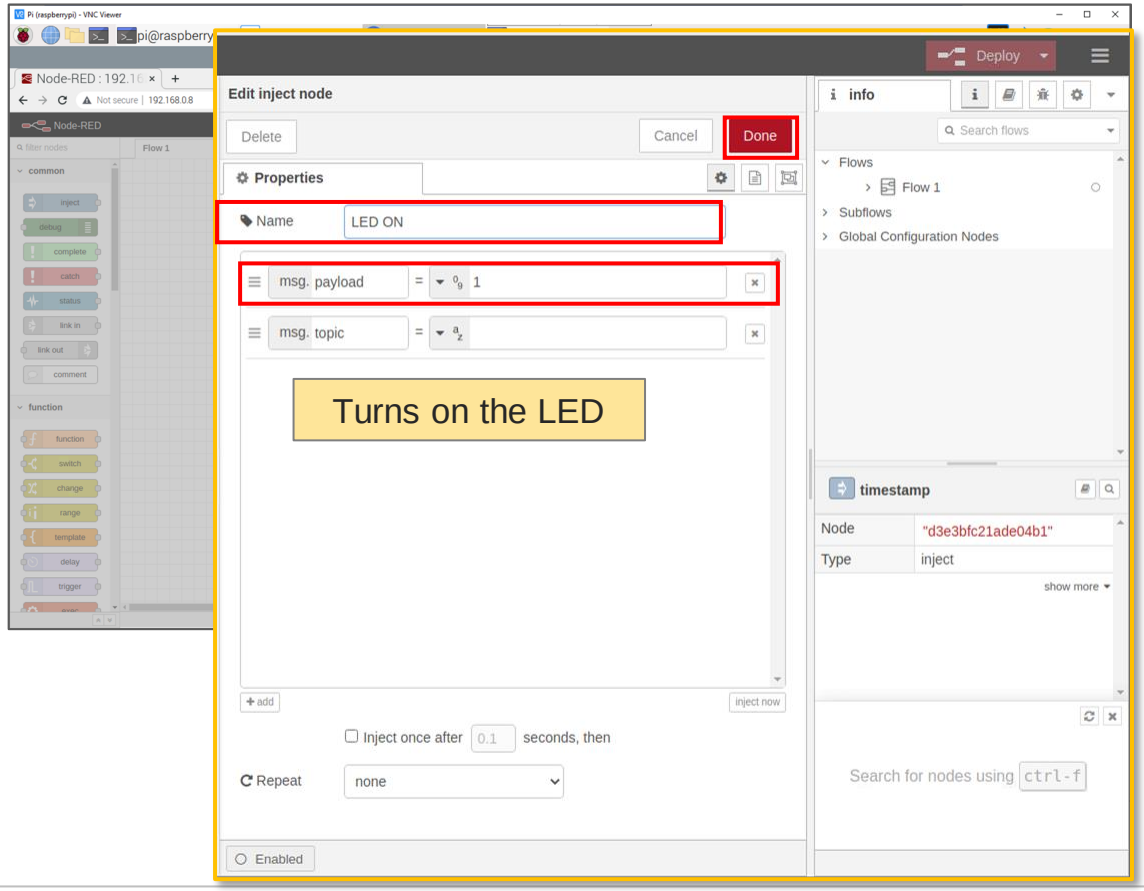
- Thêm chỉ lệnh LED ON, LED OFF



- Kéo chỉ lệnh trong danh mục chung và tạo ra hai chỉ lệnh.
- Hai chỉ lệnh được tạo ra sẽ gửi tín hiệu đến đèn LED.

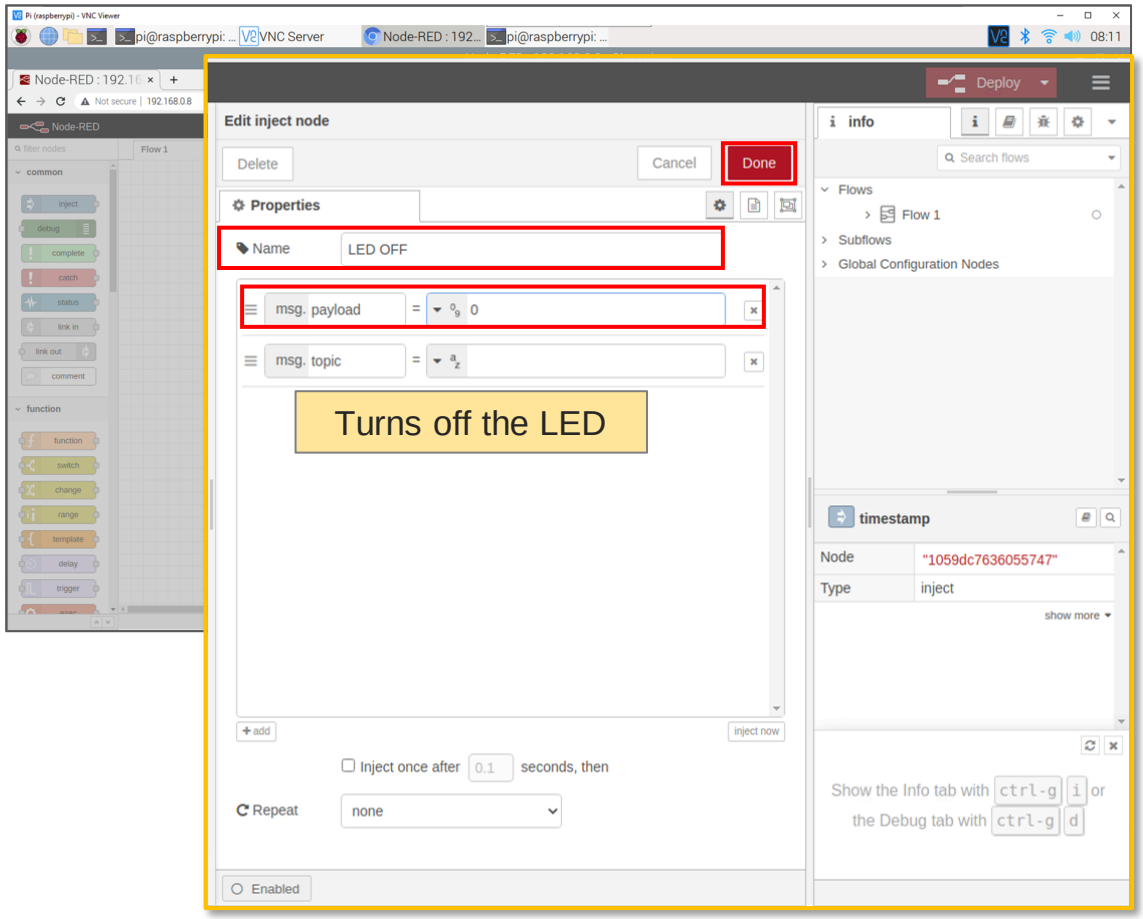
Chỉnh sửa thuộc tính của chỉ lệnh

- Chỉnh sửa thuộc tính của chỉ lệnh



- Sửa đổi các thuộc tính của chỉ lệnh đã tạo.
- Trong trường có một chỉ lệnh, Tên được đổi thành LED ON và msg.payload được đổi thành 1 của loại số Chỉ lệnh LED ON này sẽ bật đèn LED.
- Sau khi chỉnh sửa, nhấp vào nút “Done” màu đỏ.

• Chỉnh sửa thuộc tính chỉ lệnh

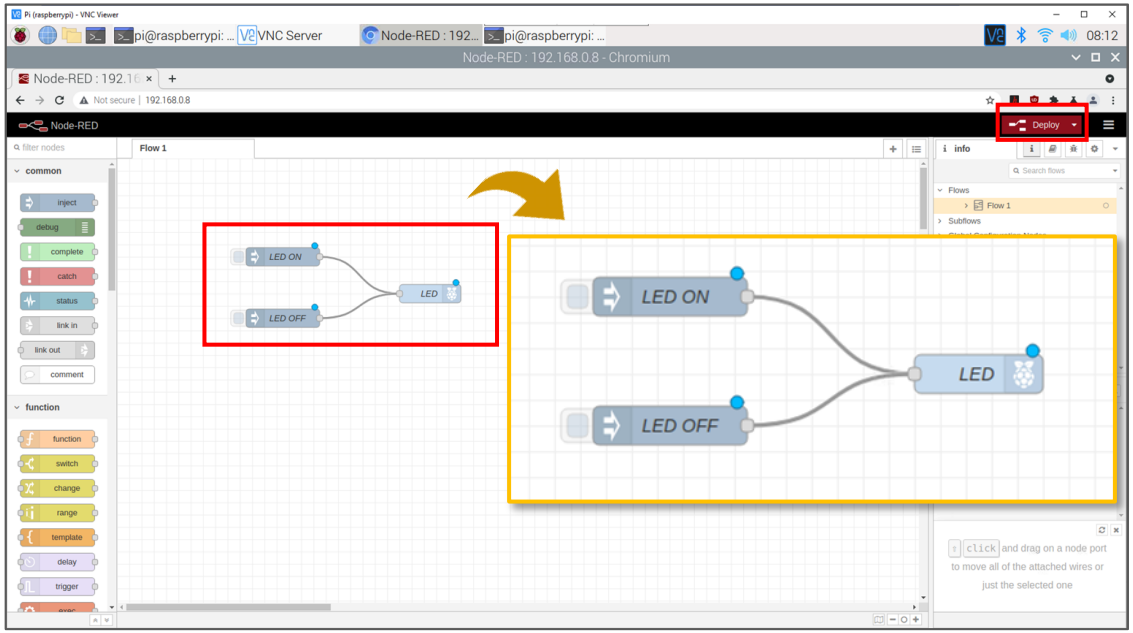


- Trong trường chỉ lệnh khác, Tên được thay đổi thành LED ON và msg.payload được thay đổi thành 0 của loại số. □ Đèn LED BẬT này sẽ tắt đèn LED.

- Sau khi chỉnh sửa, nhấp vào nút “Done” màu đỏ.

Kết nối và triển khai

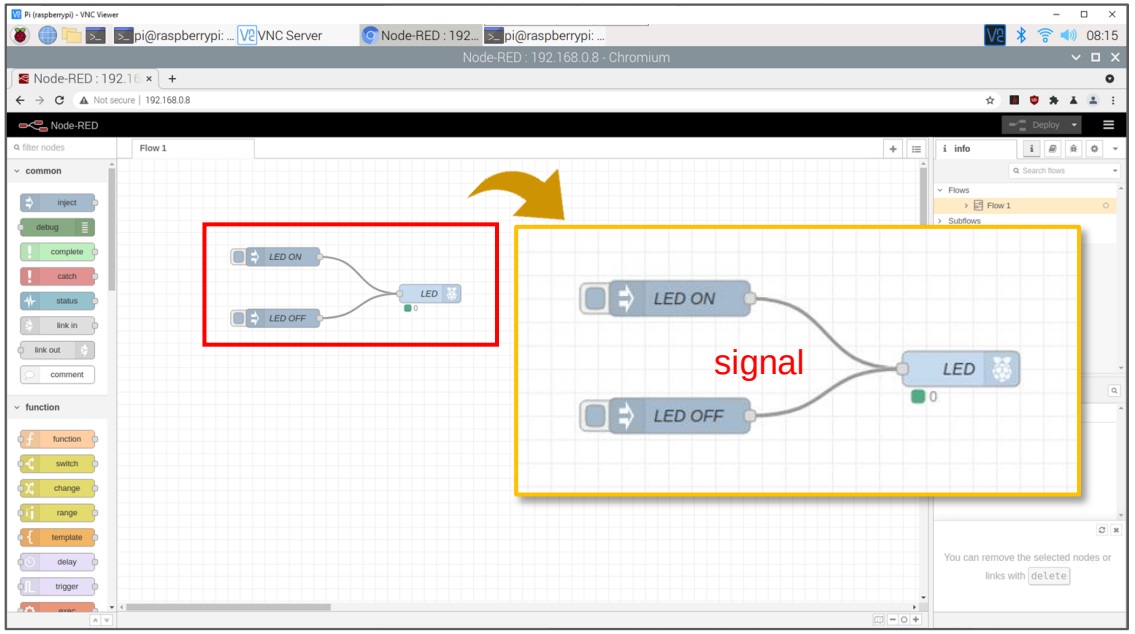
- Chỉnh sửa thuộc tính chân



- Kết nối để chỉ lệnh được tạo có thể gửi tín hiệu tới gpio.
- Sử dụng con trỏ choột để kết nối hình vuông màu xám ở bên phải của nút “LED ON” và “LED OFF” với hình vuông màu xám ở bên trái của LED.
- Vì tất cả các cài đặt đã hoàn tất, hãy nhấp vào nút “Triển khai” để chạy Flow.

Kiểm tra

- Ví dụ kiểm tra



- Sau khi nhấn nút “Deploy”, nếu bạn nhấn vào ô vuông ở bên trái nút “LED ON” và “LED OFF”, chỉ lệnh sẽ hoạt động và gửi tín hiệu đến đèn LED.
- Nhấp và kiểm tra xem đèn LED có được điều khiển hay không.

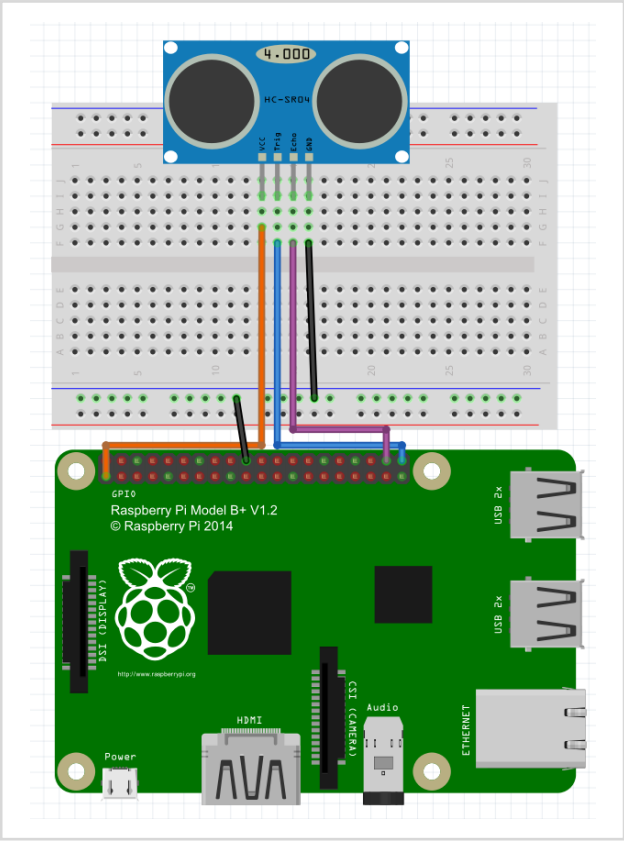
Bài 3.

Raspberry Pi với Node-RED

- 3.1. Cài đặt ban đầu
- 3.2. Điều khiển đèn LED bằng Node-RED
- 3.3. Phát hiện truy cập bằng Node-RED

Phát hiện truy cập bằng Node-RED

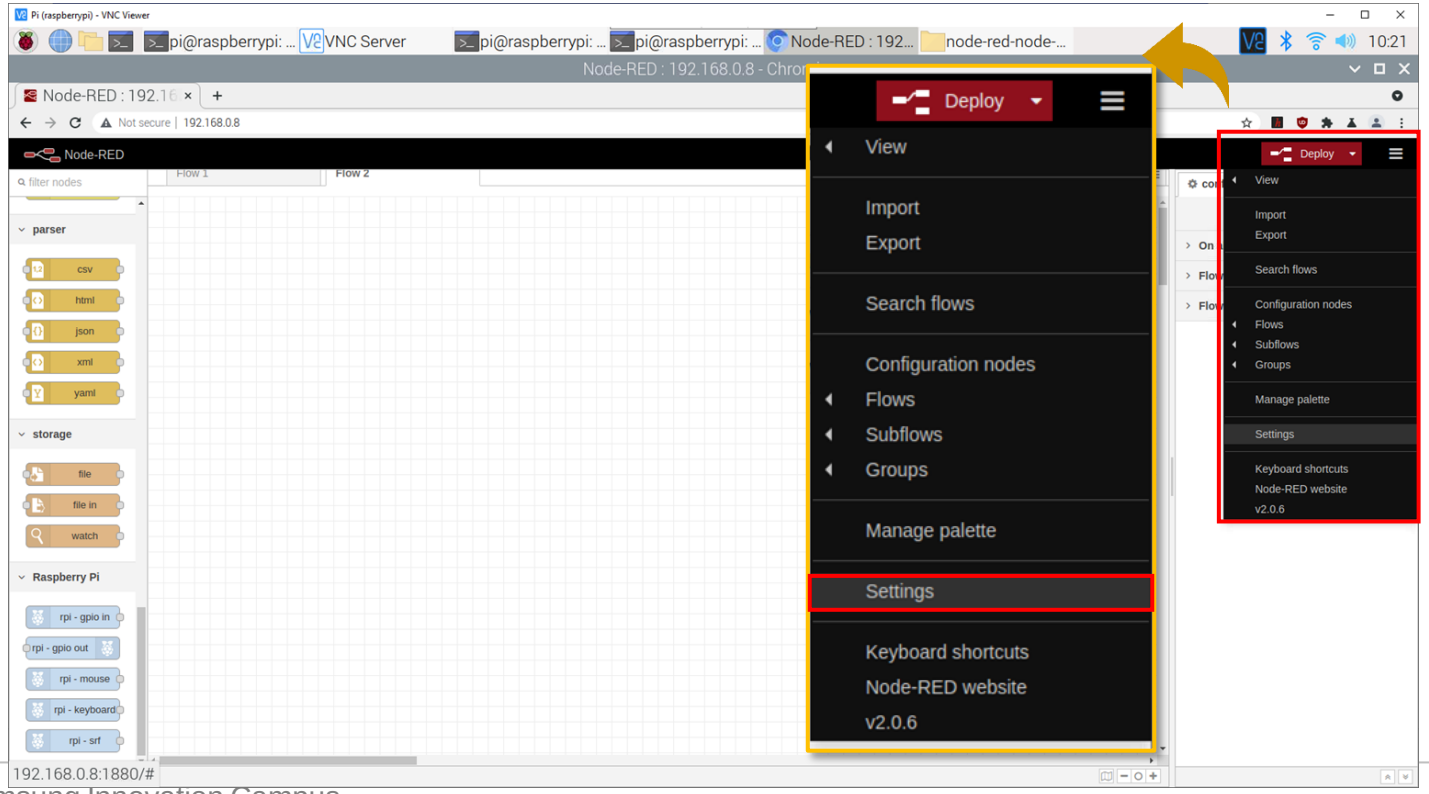
- Phát hiện truy cập bằng Node-RED



- Một ví dụ về trực quan hóa khoảng cách đo được bằng cách đo khoảng cách bằng Cảm biến siêu âm với Node-RED.
- Sơ đồ mạch
  - HC-SR04 VCC – Raspberry Pi 3.3v
  - HC-SR04 GND – GNDs
  - HC-SR04 Trig – Raspberry Pi GPIO 21
  - HC-SR04 Echo – Raspberry Pi GPIO 20
  - Raspberry Pi GND – GNDs

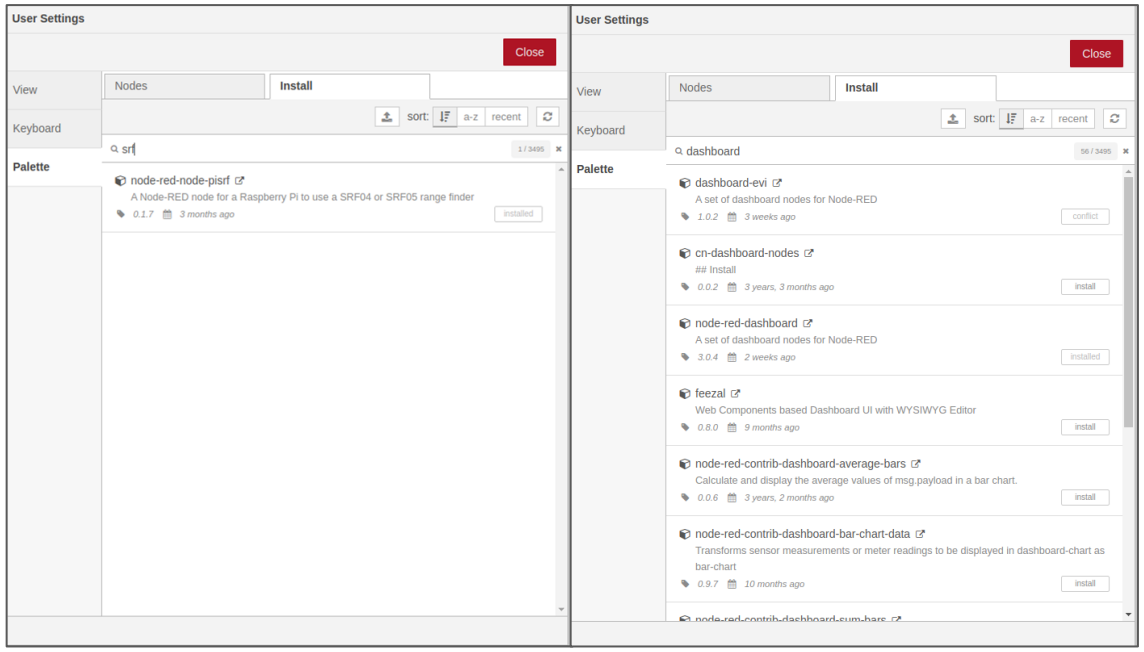
Cài đặt

- Cài đặt Nodes
  - Để cài đặt Node Cảm biến sóng Siêu âm và Node xã hội, hãy nhấn Menu ở bên phải màn hình và chọn vào Cài đặt.



Cài đặt Nodes

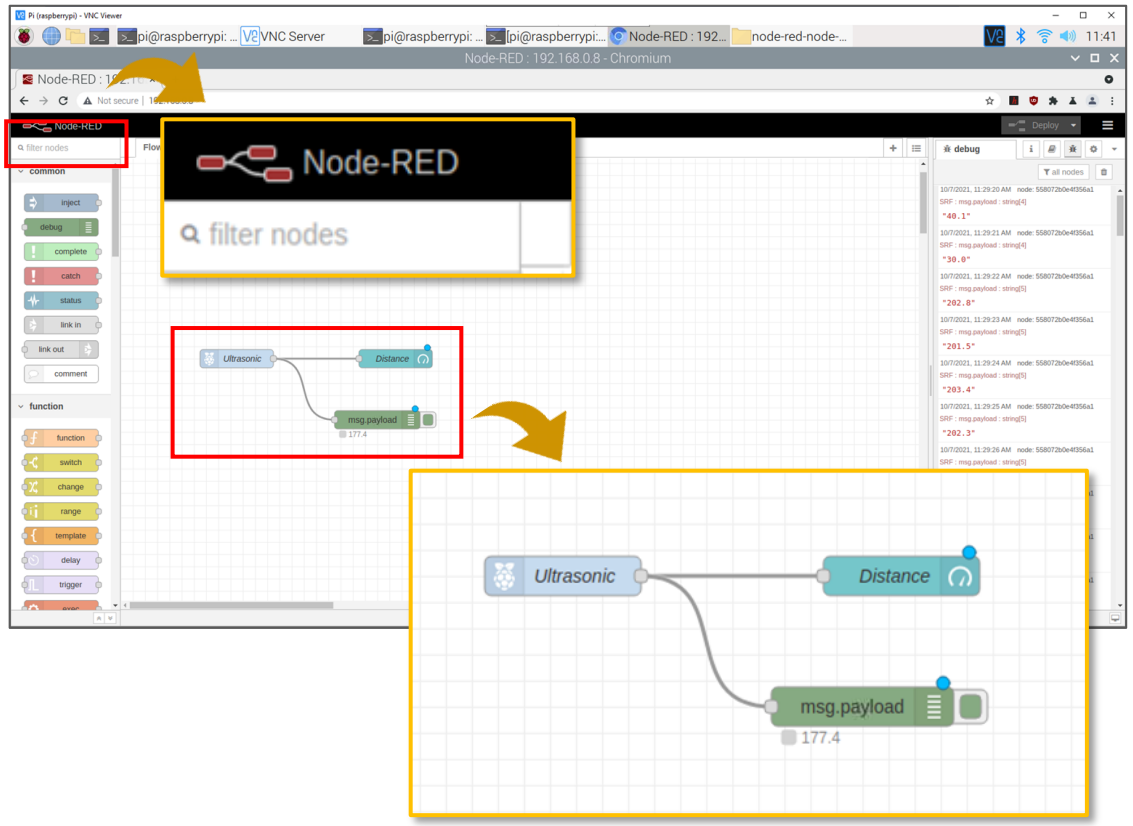
- Cài đặt node-red-node-pisrf



- Tìm srf trong tab Palette và cài đặt iperf.
- Lắp đặt cảm biến siêu âm để sử dụng trong Node-RED.
- Tìm kiếm bảng điều khiển và cài đặt node-red-dashboard.
- Cài đặt để sử dụng nút bảng điều khiển của Node-RED giúp đơn giản việc trực quan hóa.

Thêm Nodes

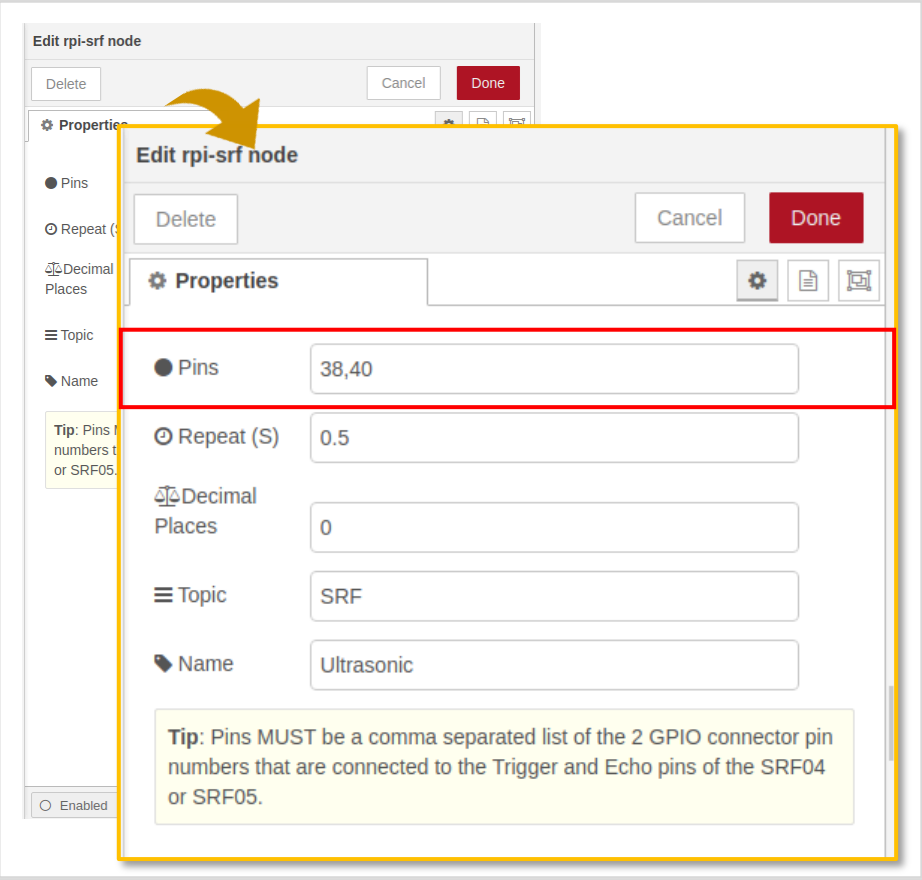
- Ví dụ bài kiểm tra



- Nhập srf, đo và gỡ lỗi trong trường nhập tìm kiếm ở trên cùng bên trái nơi ghi các nút bộ lọc, tạo từng nút một và kết nối chúng theo cách đang được thể hiện trong hình bên trái.
- Do các màu được tách rời nhau nên chỉ tiến hành sắp xếp và kết nối, còn việc cài đặt chi tiết được tiếp tục.

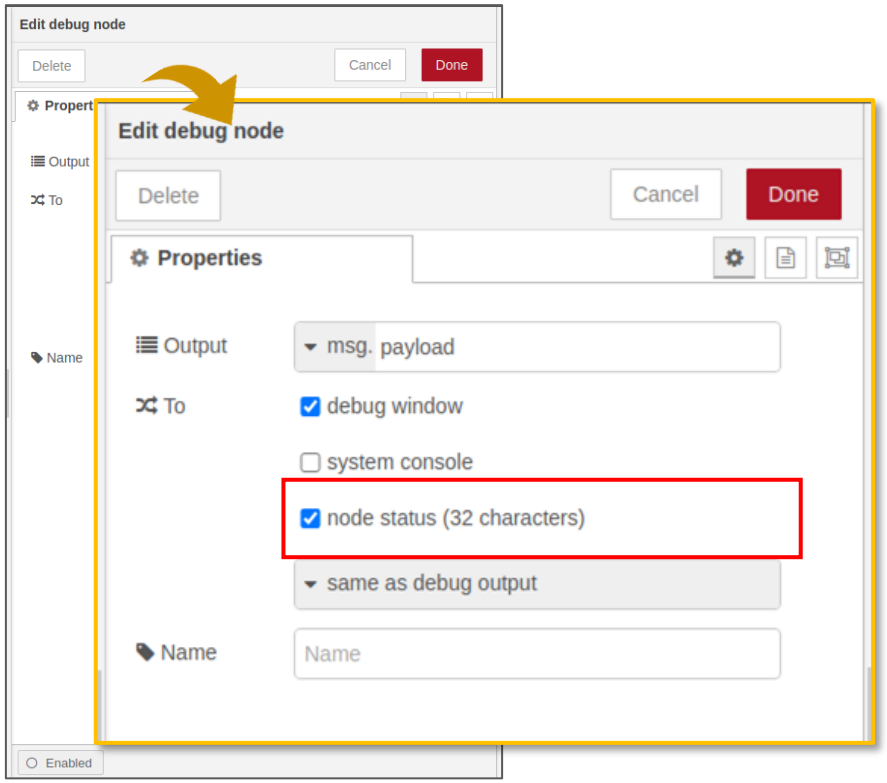
Chỉnh sửa Nodes

Chỉnh sửa thuộc tính của các node.



- rpi-srf node
  - Một điều cần lưu ý về các chân là bạn phải nhập Trig và Echo theo thứ tự và viết số của chính chân đó, không phải số chân của gpio.
  - Trig và Ccho lần lượt được kết nối với GPIO 21 và GPIO 20 và số lượng các chân này là 38 và 40.
  - Lặp lại: lặp lại chu kỳ
  - Tên: Siêu âm

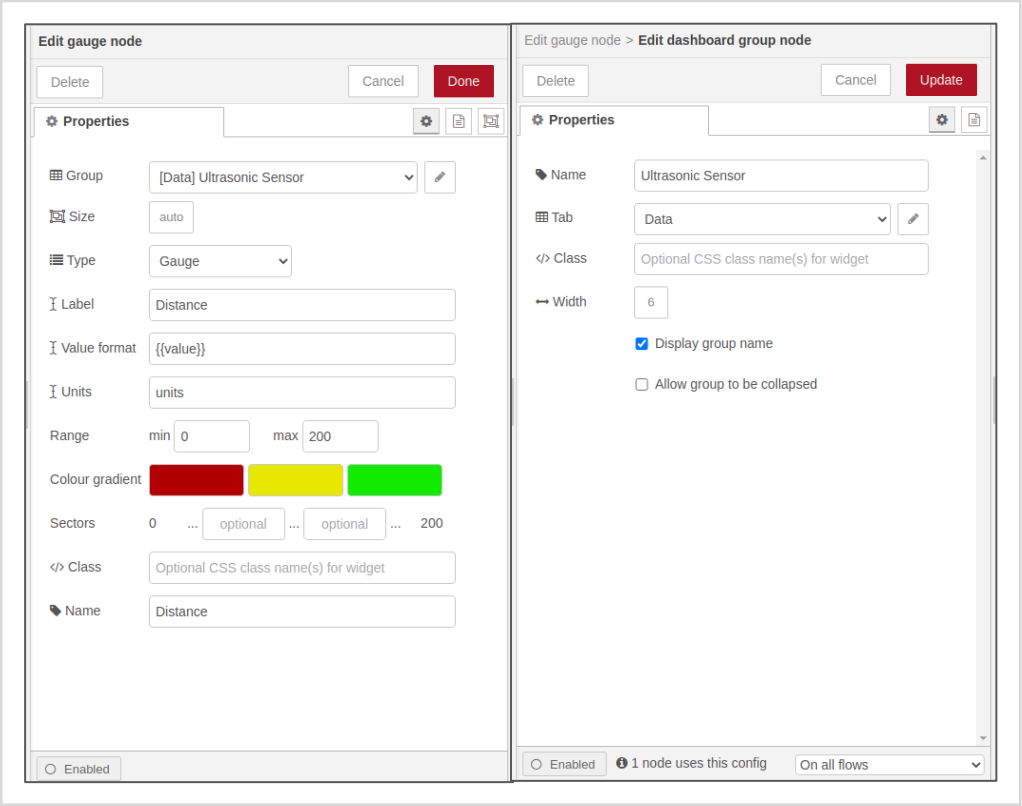
• Chỉnh sửa thuộc tính của node.



- Gỡ lỗi node
  - Kiểm tra trạng thái của node.

3.3. Phát hiện truy cập bằng Node-RED

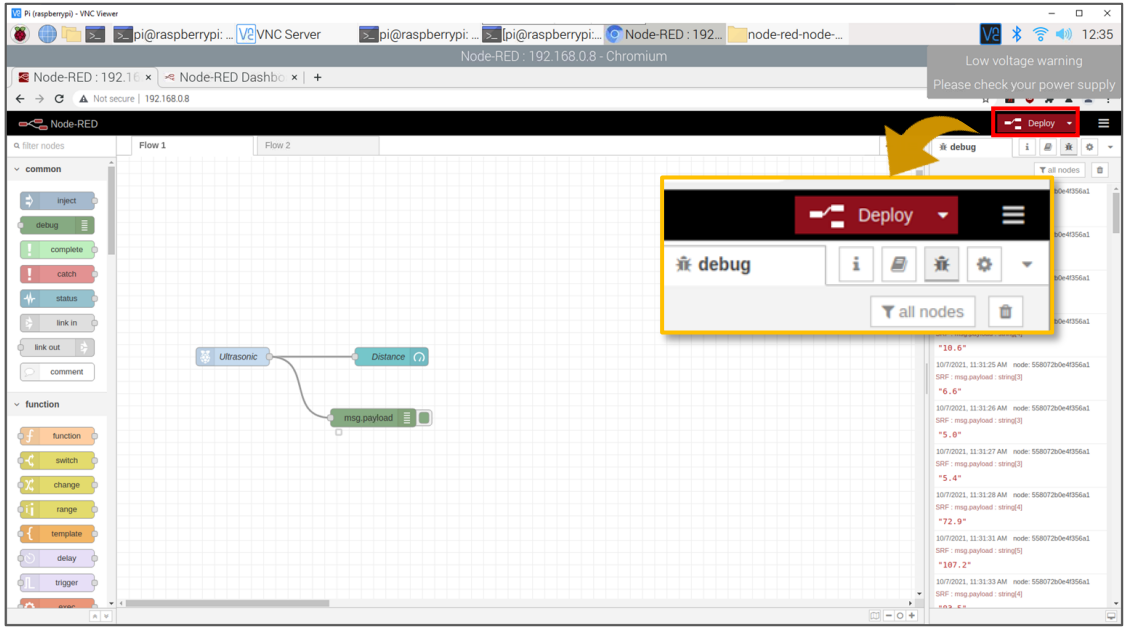
• Chỉnh sửa thuộc tính của node.



- Đo node
  - Nhóm: Chỉnh sửa thuộc tính của các node thông qua tab mở rộng và chỉnh sửa node của nhóm bảng điều khiển sao cho giống trạng thái như minh họa bên phải.
  - Nhãn : Khoảng cách
  - Phạm vi: 0 – 200
  - Thay đổi màu sắc tự do
  - Tên: Khoảng cách

Triển khai Nodes

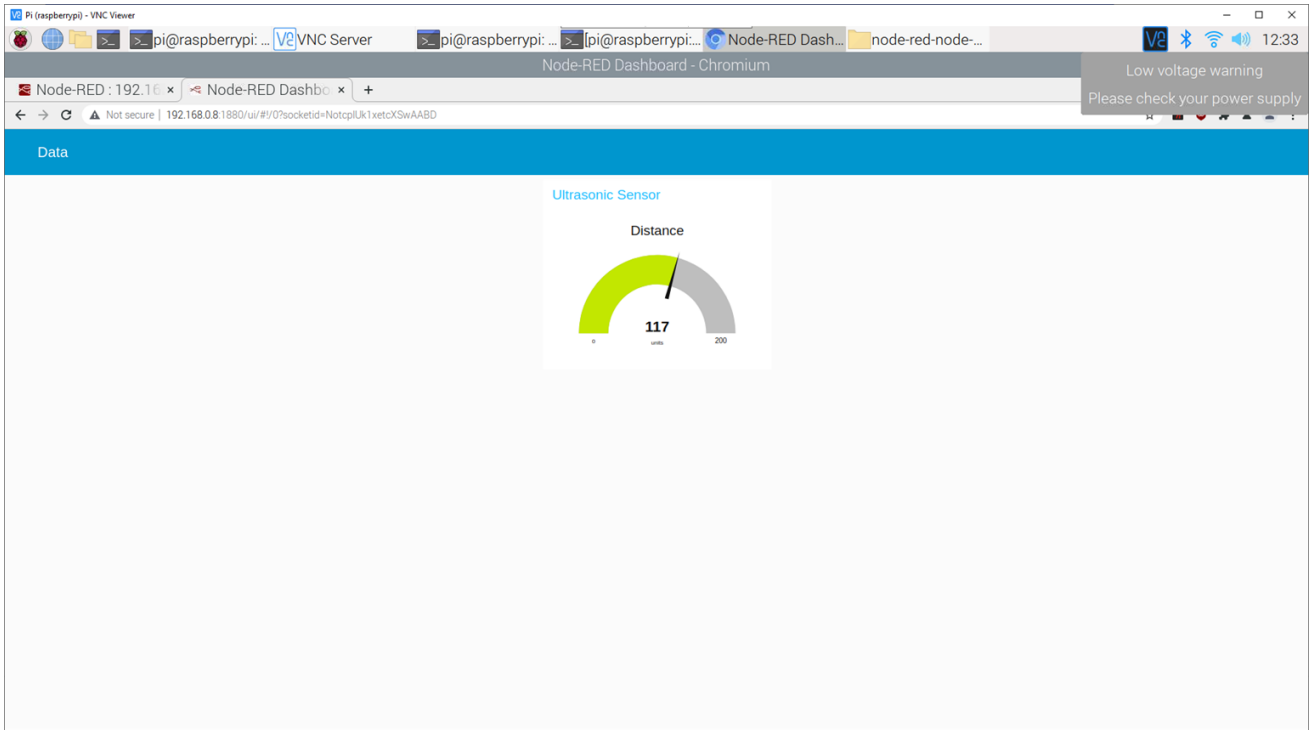
- Triển khai và kiểm tra bảng điều khiển



- Nhấp vào nút “Deploy” để khởi động quá trình.
- Nếu bạn nhấp vào Thông báo gỡ lỗi, bạn có thể thấy rằng các giá trị quan sát là các giá trị đầu ra.

Kiểm tra bảng điều khiển

- Triển khai và kiểm tra bảng điều khiển
  - Kết nối với http://Địa chỉ IP của Raspberry Pi:1880:/ui, để bạn có thể thấy thước đo thay đổi theo các phép đo của cảm biến.



Bài 4.

Xử lý lỗi và khắc phục sự cố

- 4.1. Các lỗi phổ biến nhất về cài đặt
- 4.2. Các lỗi phổ biến nhất về thông báo lỗi và lập trình
- 4.3. Sử dụng Async-Await và promises để async xử lý lỗi không đồng bộ
- 4.4. Chỉ sử dụng đối tượng Lỗi tích hợp
- 4.5. Phân biệt lỗi vận hành và lập trình viên
- 4.6. Xử lý lỗi tập trung
- 4.7. Sử dụng trình ghi nhật cập nhật để tăng khả năng hiển thị lỗi
- 4.8. Nắm bắt các chỉ lệnh promises chưa được xử lý
- 4.9. Thất bại nhanh chóng, xác thực các đối số bằng thư viện chuyên dụng

Xử lý các lỗi trong việc cài đặt

- Tìm phiên bản của gói npm đã được cài đặt
  - Cách tìm phiên bản của gói node.js/npm đã được cài đặt
  - Gói địa phương

```
Danh sách $ npm
```

  - Gói cài đặt toàn cầu

```
Danh sách $ npm
```

  - Tìm phiên bản của một gói cụ thể bằng cách chuyển tên của nó làm đối số

```
Danh sách $ npm grunt
```

- Cách cài đặt phiên bản chính xác trước đó của gói NPM
  - Nếu bạn phải cài đặt phiên bản cũ hơn của gói, hãy làm rõ nó.

```
Cài đặt $ npm <package>@<version>
```

x

```
Cài đặt $ npm express@3.0.0
```

  - Bạn cũng có thể thêm cờ --save vào lệnh đó để thêm nó vào phần phụ thuộc pack.json của bạn hoặc cờ --save --save-exact nếu bạn muốn phiên bản chính xác được chỉ định trong phần phụ thuộc pack.json của mình.
  - Nếu bạn không chắc có những phiên bản nào của gói, bạn có thể sử dụng:

```
$ npm view <package> versions
```

- Thông báo“ Message failed to fetch from registry" trong khi cố gắng cài đặt mô-đun bất kỳ
  - Nếu bạn gặp vấn đề với việc cài đặt **npm {node module bất kỳ}**
  - Cập nhật node (và npm) giải quyết vấn đề.
  - Đầu tiên, hãy gỡ cài đặt phiên bản lỗi thời (tùy chọn, điều này đã khắc phục sự cố gặp phải với các mô-đun chung không được dẫn vào).

```
$ sudo apt-get purge nodejs npm
```

  - Sau đó bật repo của nodesource và cài đặt.

```
$ curl -sL https://deb.nodesource.com/setup | sudo bash -
$ sudo apt-get install -y nodejs
```

- Khi cố gắng tìm kiếm hoặc cài đặt thứ gì đó bằng npm, sẽ xảy ra lỗi sau nếu bạn không sử dụng lệnh ‘sudo’ ngay cả khi bạn đã là quản trị viên.
  - Đây có vẻ như là sự cố về quyền trong thư mục chính của bạn. Để đòi lại quyền sở hữu thư mục .npm, hãy thực thi.

```
$ sudo chown -R $(whoami) ~/.npm
```

  - Ngoài ra, nếu bắt buộc, bạn sẽ cần quyền ghi trong thư mục node_modules.

```
$ sudo chown -R $USER /usr/local/lib/node_modules
```

- npm WARN chưa đáp ứng được sự phụ thuộc (unmet dependency)
  - Sau đây là những giải pháp có thể
    - Cần cài đặt các mô-đun cấp cao nhất theo cách thủ công, chứa các unmet dependency: cài đặt npm **findup-sync@0.1.2**
    - Cấu trúc lại package.json. Đặt tất cả các module cấp cao (đóng vai trò phụ thuộc cho các module khác) ở dưới cùng.
    - Chạy lại lệnh cài đặt **npm**.
  - Sự cố có thể do npm không thể tải xuống tất cả các gói do hết thời gian chờ hoặc do nguyên nhân khác
  - Bạn có thể cài đặt các gói bị lỗi theo cách thủ công bằng cách sử dụng npm install **findup-sync@0.1.2.**
  - Nếu bạn gặp thông báo lỗi như thế này khi cố gắng cài đặt npm từ gói gốc.

```
npm WARN unmet dependency /Users/seanmackesey/google_drive/code/explore/generator/node_modules/findup-sync/node_modules/glob requires graceful-fs@'~1.2.0' but will load
```

  - Khả năng do độ phân giải phụ thuộc bị hỏng nhẹ, xem

https://github.com/npm/npm/issues/1341#issuecomment-20634338

Bài 4.

Xử lý lỗi và khắc phục sự cố

- 4.1. Các lỗi phổ biến nhất về cài đặt
- 4.2. Các lỗi phổ biến nhất về thông báo lỗi và lập trình
- 4.3. Sử dụng Async-Await và promises để async xử lý lỗi không đồng bộ
- 4.4. Chỉ sử dụng đối tượng Lỗi tích hợp
- 4.5. Phân biệt lỗi vận hành và lập trình viên
- 4.6. Xử lý lỗi tập trung
- 4.7. Sử dụng trình ghi nhật ký cập nhật để tăng khả năng hiển thị lỗi
- 4.8. Nắm bắt các chỉ lệnh promises chưa được xử lý
- 4.9. Thất bại nhanh chóng, xác thực các đối số bằng thư viện chuyên dụng

Xử lý lỗi về cài đặt

- Tìm phiên bản của gói npm đã cài đặt.
  - Sử dụng Node.js **console.time()** và **console.timeEnd()**

x

```
var i;
console.time( label: "dbsave");

for(i = 1; i < LIMIT; i++){
    db.users.save({id : i, name : "MongoUser [" + i + "]"}, end);
}

end = function(err, saved) {
    console.log(( err || !saved )?"Error":"Saved");
    if(--i === 1){console.timeEnd( label: "dbsave");}
};
```

- in dấu vết ngăn xếp trong Node.js
  - . Bất cứ đối tượng Error cũng chứa phần tử ngăn xếp có tác dụng dừng con trỏ tại vị trí mà nó được gọi tớ.

```
var stack = new Error().stack
console.log( stack )
```

  - hoặc đơn giản hơn như dưới đây.

```
console.trace( data: "Here I am!")
```

- Lỗi: EACCES: quyền bị từ chối, quyền truy cập '/usr/local/lib/node_modules'(1)
  - Thay đổi quyền của bạn
  - Trước tiên, hãy kiểm tra xem ai sở hữu thư mục.

```
$ ls -la /usr/local/lib/node_modules
```

  - nó đang từ chối quyền truy cập vì thư mục node_module được sở hữu bởi root.

```
drwxr-xr-x   3 root    wheel  102 Jun 24 23:24 node_modules
```

  - Vì vậy, điều này cần được thay đổi bằng cách thay đổi quyền root thành người dùng của bạn, nhưng trước tiên, hãy chạy lệnh bên dưới để kiểm tra tình trạng người dùng hiện tại Làm cách nào để biết tên của người dùng đang hoạt động thông qua dòng lệnh trong OS X?

- Lỗi: EACCES: quyền bị từ chối, quyền truy cập '/usr/local/lib/node_modules'(2)
  - **id –un** HOẶC **whoami**

```
$ id -un
```

```
$ whoami
```

  - Thay đó, thay đổi người sở hữu

```
$ sudo chown -R [owner]:[owner] /usr/local/lib/node_modules
```

Hoặc

```
$ sudo chown -R ownerName: /usr/local/lib/node_modulest
```

- Mã lỗi ENOSPC
  - “ENOSPC" có nghĩa là không có dung lượng trống trên ổ đĩa.
  - Chạy lệnh dưới đây để tránh ENOSPC.

```
$ echo fs.inotify.max_user_watches=524288 | sudo tee -a /etc/sysctl.conf && sudo sysctl -p
```

  - Đối với Arch Linux, hãy thêm lệnh bên dưới vào /etc/sysctl.d/99-sysctl.conf.

```
fs.inotify.max_user_watches=524288
```

  - Sau đó, thực hiện:

```
$ sysctl --system
```

- Lỗi: Không thể đặt tiêu đề sau khi được gửi tới máy khách hàng
  - Khắc phục sự cố này hoạt động bNode.js 4.10 và Express 2.4.3.
  - Đối tượng **res** trong Express là một tập con của **httpServerResponse**. của Node.js (đọc nguồn http.js). Bạn được phép gọi **res.setHeader(tên, giá trị)** bao nhiêu lần tùy thích cho đến khi bạn gọi **res.writeHead(statusCode)**. Sau **writeHead**, các tiêu đề được đưa vào và bạn chỉ có thể gọi **res.write(data)** và cuối cùng là **res.end(data).**
  - Lỗi "Lỗi: Không thể đặt tiêu đề sau khi được gửi." có nghĩa là bạn đã ở trạng thái Đang thực hiện hoặc Đã hoàn thành, nhưng một số chức năng đã cố đặt tiêu đề hoặc Mã trạng thái. Khi bạn thấy lỗi này, hãy cố gắng tìm kiếm bất kỳ thứ gì cố gắng gửi tiêu đề sau khi một phần nội dung đã được viết.

    x tìm kiếm các Callbacks lại vô tình được gọi hai lần hoặc bất kỳ lỗi nào xảy ra sau khi nội dung được gửi.

- Cách khắc phục lỗi npm "npm ERR! code ELIFECYCLE"
  - Nếu bạn gặp mã lỗi ELFECYCLE khi chạy npm start:

Bước 1

```
$ npm cache clean --force
```

  - Bước 2: Xóa thư mục node_modules bằng thư mục $ rm -rf node_modules (rmdir /S /Q node_modules trong windows) hoặc xóa thủ công bằng cách vào thư mục. Nhấp chuột phải > xóa/di chuyển vào thùng rác. Nếu không cập nhật các gói của mình, bạn có thể xóa tệp pack-lock.json.

```
$ rm -rf node_modules package-lock.json
```

  - Bước 3

```
$ npm install
```

  - Bước 4: Thử lại

```
$ npm start
```

- SyntaxError: Sử dụng const trong chế độ nghiệm ngặt
  - Chúng tôi đang làm việc trong node.js và chúng tôi đang sử dụng nó giới hạn ở "chế độ nghiêm ngặt" trong một trong các tệp js. Có lỗi xảy ra khi cố gắng thực thi.
  - Answer Case 1 : Updating nodejs solved the issue:

```
npm cache clean - f
sudo npm install - g n
sudo n stable            #or, $nvm install stable
node--version
node app.js
```

Cập nhật NodeJS giải quyết vấn đề này. Tuy nhiên, sau khi chạy **$sudo npm install -g n** , bạn có thể gặp lỗi sau:

npm: tái định vị error: npm: symbol SSL_set_cert_cb, phiên bản libssl.so .10 không được xác định trong tập tin libssl.so .10 với thời gian tham chiếu

Để khắc phục lỗi này, hãy thử nâng cấp openssl bằng lệnh bên dưới:

```
sudo yum update openssl
```

  - Đáp án trường hợp 2:

```
'use strict'
const MAX_IMAGE_SIZE = 1024*1024; // 1 MB
```

- Chạy Node.js như một tiến trình nền không bao giờ chết
  - Nếu bạn đã cố chạy nó dưới dạng một quy trình nền như:

```
$ node server.js &
```

  - Khi thiết bị đầu cuối không hoạt động và quá trình chết.
  - Bạn muốn tìm một cách có thể duy trì tiến trình ngay cả khi thiết bị đầu cuối bị ngắt kết nối.

```
$ nohup node server.js > /dev/null 2>&1 &
```

    - nohup: Không chấm dứt quá trình này ngay cả khi stty bị cắt.
    - > /dev/null: stdout đi tới/dev/null (đó là một thiết bị giả không ghi lại bất kỳ kết quả nào).
    - 2>&1: stderr also goes to the stdout (which is already redirected to /dev/null). Bạn có thể thay thế &1 bằng đường dẫn tệp để ghi nhật ký lỗi).
    - &: chạy lệnh này như một tác vụ nền.

Bài 4.

Xử lý lỗi và khắc phục sự cố

- 4.1. Các lỗi phổ biến nhất về cài đặt
- 4.2. Các lỗi phổ biến nhất về thông báo lỗi và lập trình
- 4.3. Sử dụng Async-Await và promises để async xử lý lỗi không đồng bộ
- 4.4. Chỉ sử dụng đối tượng Lỗi tích hợp
- 4.5. Phân biệt lỗi vận hành và lập trình viên
- 4.6. Xử lý lỗi tập trung
- 4.7. Sử dụng trình ghi nhật ký cập nhật để tăng khả năng hiển thị lỗi
- 4.8. Nắm bắt các chỉ lệnh promises chưa được xử lý
- 4.9. Thất bại nhanh chóng, xác thực các đối số bằng thư viện chuyên dụng

Xử lý lỗi không đồng bộ

- Các Callbacks không mở rộng quy mô tốt vì hầu hết các lập trình viên không quen thuộc với chúng.
- Họ nhìn thấy toàn bộ lỗi, xử lý việc lồng mã phức tạp và gây phiền toái cho việc suy luận về dòng mã.
- Các thư viện Promise như BlueBird, async và Q sử dụng RETURN và THROW để nén các kiểu mã tiêu chuẩn kiểm soát luồng chương trình.
- Đặc biệt, nó hỗ trợ kiểu xử lý lỗi bắt thử được yêu thích, kiểu này ngăn đường dẫn mã mặc định trong việc xử lý lỗi cho tất cả các chức năng.

- Ví dụ mã - sử dụng Promise để bắt lỗi

```
return functionA()
    .then(functionB)
    .then(functionC)
    .then(functionD)
    .catch((err) => logger.error(err))
    .then(alwaysExecuteThisFunction)
```

• Ví dụ về mã - sử dụng async/await để bắt lỗi

```
async function executeAsyncTask () {
    try {
        const valueA = await functionA();
        const valueB = await functionB(valueA);
        const valueC = await functionC(valueB);
        return await functionD(valueC);
    }
    catch (err) {
        logger.error(err);
    } finally {
        await alwaysExecuteThisFunction();
    }
}
```

- Ví dụ mã chống mẫu - xử lý lỗi kiểu Callbacks

```
getData(someParameter, function(err, result) {
    if(err !== null) {
        // do something like
        // calling the given callback function
        // and pass the error
        getMoreData(a, function(err, result) {
            if(err !== null) {
                // do something like
                // calling the given callback function
                // and pass the error
                getMoreData(b, function(c) {
                    getMoreData(d, function(e) {
                        if(err !== null ) {
                            // you get the idea?
                        }
                    })
                });
            }
        });
    }
});
```

Bài 4.

Xử lý lỗi và khắc phục sự cố

- 4.1. Các lỗi phổ biến nhất về cài đặt
- 4.2. Các lỗi phổ biến nhất về thông báo lỗi và lập trình
- 4.3. Sử dụng Async-Await và promises để async xử lý lỗi không đồng bộ
- 4.4. Chỉ sử dụng đối tượng Lỗi tích hợp
- 4.5. Phân biệt lỗi vận hành và lập trình viên
- 4.6. Xử lý lỗi tập trung
- 4.7. Sử dụng trình ghi nhật ký cập nhật để tăng khả năng hiển thị lỗi
- 4.8. Nắm bắt các chỉ lệnh promises chưa được xử lý
- 4.9. Thất bại nhanh chóng, xác thực các đối số bằng thư viện chuyên dụng

Đối tượng lỗi tích hợp

- Bản chất dễ tiếp cận của JavaScript và các tùy chọn luồng mã khác nhau (chẳng hạn như EventEmitterCallbacks, Promises v.v.) thay đổi cách các nhà phát triển đưa ra lỗi. Một số sử dụng chuỗi, những người khác xác định các loại tùy chỉnh của riêng họ.
- Đối tượng Lỗi tích hợp sẵn của Node.js cho phép bạn duy trì tính đồng nhất trong mã của mình và cũng bảo toàn thông tin quan trọng như StackTrace cùng với các thư viện của bên thứ ba.
- Khi đưa ra một ngoại lệ, bạn nên đưa vào ngoại lệ các thuộc tính ngữ cảnh bổ sung, chẳng hạn như tên lỗi và mã lỗi HTTP liên quan.
- Để đạt được sự thống nhất và thông lệ này, hãy xem xét việc mở rộng đối tượng Lỗi bằng các thuộc tính bổ sung, nhưng hãy cẩn thận không lạm dụng nó.
- Nói chung, bạn chỉ nên mở rộng đối tượng Lỗi tích hợp sẵn một lần với AppError cho tất cả các lỗi cấp ứng dụng và chuyển dữ liệu cần thiết làm đối số để phân biệt giữa các loại lỗi khác nhau.
- Không cần phải mở rộng đối tượng Lỗi nhiều lần (một lần cho từng trường hợp lỗi như DbError, HttpError).

- Ví dụ mã – làm đúng

```
// throwing an Error from typical function,
// whether sync or async
if(!productToAdd)
    throw new Error('How can I add new product ' +
        'when no value provided?');

// 'throwing' an Error from EventEmitter
const myEmitter = new MyEmitter();
myEmitter.emit('error', new Error('whoops!'));

// 'throwing' an Error from a Promise
const addProduct = async (productToAdd) => {
    try {
        const existingProduct =
            await DAL.getProduct(productToAdd.id);
        if (existingProduct !== null) {
            throw new Error('Product already exists!');
        }
    } catch (err) {
        // ...
    }
}
```

Ví dụ mã– Anti Pattern

```
// throwing a string lacks any stack trace information
// and other important data properties
if(!productToAdd)
    throw ('How can I add new product when no value provided?');
```

- Ví dụ mã - làm điều đó thậm chí còn tốt hơn

```
// centralized error object that derives from Node's Error
function AppError(name, httpCode, description, isOperational) {
    Error.call(this);
    Error.captureStackTrace(this);
    this.name = name;
    //...other properties assigned here
};

AppError.prototype = Object.create(Error.prototype);
AppError.prototype.constructor = AppError;

module.exports.AppError = AppError;

// client throwing an exception
if(user == null)
    throw new AppError(
        commonErrors.resourceNotFound,
        commonHTTPErrors.notFound,
        description: 'further explanation',
        isOperational: true)
```

Bài 4.

Xử lý lỗi và khắc phục sự cố

- 4.1. Các lỗi phổ biến nhất về cài đặt
- 4.2. Các lỗi phổ biến nhất về thông báo lỗi và lập trình
- 4.3. Sử dụng Async-Await và promises để async xử lý lỗi không đồng bộ
- 4.4. Chỉ sử dụng đối tượng Lỗi tích hợp
- 4.5. Phân biệt lỗi vận hành và lập trình viên
- 4.6. Xử lý lỗi tập trung
- 4.7. Sử dụng trình ghi nhật cập nhật để tăng khả năng hiển thị lỗi
- 4.8. Nắm bắt các chỉ lệnh promises chưa được xử lý
- 4.9. Thất bại nhanh chóng, xác thực các đối số bằng thư viện chuyên dụng

Lỗi Vận hành & Lập trình

- Phân biệt hai loại lỗi này sẽ giúp bạn giảm thiểu thời gian ngừng hoạt động của ứng dụng và tránh các lỗi phức tạp: Lỗi vận hành đề cập đến các tình huống mà bạn hiểu điều gì đã xảy ra và tác động của nó. Ví dụ: một truy vấn tới một số dịch vụ HTTP không thành công do sự cố kết nối.
- Mặt khác, lỗi lập trình đề cập đến các trường hợp bạn không biết tại sao và đôi khi chúng đến từ đâu. Đó có thể là một số mã đang cố đọc nhóm kết nối DB với các giá trị không xác định hoặc rò rỉ bộ nhớ.
- Các lỗi vận hành tương đối dễ xử lý. Thường là lỗi đăng nhập.
- Khi lỗi lập trình viên xảy ra, mọi thứ trở nên phức tạp. Ứng dụng của bạn có thể ở trạng thái không nhất quán và không có gì tốt hơn là khởi động lại.

- Ví dụ về mã – đánh dấu lỗi thuộc về vận hành (đáng tin cậy)

```
// marking an error object as operational
const myError = new Error('How can I add new product ' +
    'when no value provided?');
myError.isOperational = true;

// or if you're using some centralized error factory
// (see other examples at the bullet
// "Use only the built-in Error object")
class AppError {
    constructor (commonType, description, isOperational) {
        Error.call(this);
        Error.captureStackTrace(this);
        this.commonType = commonType;
        this.description = description;
        this.isOperational = isOperational;
    }
};

throw new AppError(
    errorManagement.commonErrors.InvalidInput,
    description: 'Describe here what happened',
    isOperational: true);
```

Bài 4.

Xử lý lỗi và khắc phục sự cố

- 4.1. Các lỗi phổ biến nhất về cài đặt
- 4.2. Các lỗi phổ biến nhất về thông báo lỗi và lập trình
- 4.3. Sử dụng Async-Await và promises để async xử lý lỗi không đồng bộ
- 4.4. Chỉ sử dụng đối tượng Lỗi tích hợp
- 4.5. Phân biệt lỗi vận hành và lập trình viên
- 4.6. Xử lý lỗi tập trung
- 4.7. Sử dụng trình ghi nhật ký cập nhật để tăng khả năng hiển thị lỗi
- 4.8. Nắm bắt các chỉ lệnh promises chưa được xử lý
- 4.9. Thất bại nhanh chóng, xác thực các đối số bằng thư viện chuyên dụng

Không nằm trong phần mềm trung gian

- Nếu không có một đối tượng chuyên dụng để xử lý lỗi, khả năng xử lý lỗi không nhất quán sẽ tăng lên.
- Lỗi xảy ra trong yêu cầu web có thể được xử lý khác với lỗi xảy ra trong giai đoạn khởi động và lỗi xảy ra trong các tác vụ đã được lên kế hoạch.
- Điều này có thể dẫn đến một số loại lỗi được quản lý sai.
- Đối tượng trình xử lý lỗi duy nhất này làm cho lỗi hiển thị bằng cách ghi vào trình nhật ký trực quan, kích hoạt số liệu bằng cách sử dụng một số sản phẩm giám sát (như Prometheus, CloudWatch, DataDog và Sentry) và quyết định xem quy trình có bị lỗi hay không.
- Hầu hết các khung web đều cung cấp cơ chế phần mềm trung gian bắt lỗi. Một lỗi phổ biến là mã xử lý lỗi xuất hiện trong phần mềm trung gian.
- Điều này đảm bảo rằng không thể sử dụng lại cùng một trình xử lý cho các lỗi gặp phải trong các tình huống khác nhau, chẳng hạn như các tác vụ đã lên lịch, người đăng ký hàng đợi tin nhắn và các ngoại lệ chưa được phát hiện.
- Phần mềm trung gian lỗi chỉ nên bắt lỗi và chuyển nó cho trình xử lý.
- Quy trình xử lý lỗi điển hình như sau: Một số mô-đun đưa ra lỗi → bộ định tuyến API bắt lỗi → truyền lỗi đến phần mềm trung gian chịu trách nhiệm bắt lỗi (ví dụ: các cơ chế khác để bắt lỗi cấp độ yêu cầu) → trình xử lý lỗi tập trung được sử dụng.

Xử lý lỗi tập trung

- Ví dụ mã - một luồng lỗi điển hình

```
// DAL layer, we don't handle errors here
DB.addDocument(newCustomer, (error, result) => {
    if (error)
        throw new Error('Great error explanation comes here',
            other useful parameters)
});

// API route code, we catch both sync and async errors and
// forward to the middleware
try {
    customerService.addNew(req.body).then((result) => {
        res.status(200).json(result);
    }).catch((error) => {
        next(error)
    });
}
catch (error) {
    next(error);
}
```

```
// Error handling middleware, we delegate the handling
// to the centralized error handler
app.use(async (err, req, res, next) => {
    await errorHandler.handleError(err, res);
    //The error handler will send a response
});

process.on("uncaughtException", error => {
    errorHandler.handleError(error);
});

process.on("unhandledRejection", (reason) => {
    errorHandler.handleError(reason);
});
```

- Ví dụ mã - xử lý lỗi trong một đối tượng chuyên dụng

```
module.exports.handler = new errorHandler();

function errorHandler() {
    this.handleError = async (error, responseStream) => {
        await logger.logError(error);
        await fireMonitoringMetric(error);
        await crashIfUntrustedErrorOrSendResponse(
            error, responseStream);
    };
}
```

• Ví dụ mã – Anti Pattern: xử lý lỗi trong phần mềm trung gian

```
// middleware handling the error directly,
// who will handle Cron jobs and testing errors?
app.use((err, req, res, next) => {
    logger.logError(err);
    if (err.severity == errors.high) {
        mailer.sendMail(configuration.adminMail,
            'Critical error occured', err);
    }
    if (!err.isOperational) {
        next(err);
    }
});
```

Bài 4.

Xử lý lỗi và khắc phục sự cố

- 4.1. Các lỗi phổ biến nhất về cài đặt
- 4.2. Các lỗi phổ biến nhất về thông báo lỗi và lập trình
- 4.3. Sử dụng Async-Await và promises để async xử lý lỗi không đồng bộ
- 4.4. Chỉ sử dụng đối tượng Lỗi tích hợp
- 4.5. Phân biệt lỗi vận hành và lập trình viên
- 4.6. Xử lý lỗi tập trung
- 4.7. Sử dụng trình ghi nhật ký cập nhật để tăng khả năng hiển thị lỗi
- 4.8. Nắm bắt các chỉ lệnh promises chưa được xử lý
- 4.9. Thất bại nhanh chóng, xác thực các đối số bằng thư viện chuyên dụng

sử dụng Logger

- Chúng tôi yêu thích console.log nhưng một trình ghi nhật ký bền bỉ và có uy tín như Pino (một tùy chọn mới hơn tập trung vào hiệu suất) là bắt buộc đối với các dự án nghiêm túc.
- Các công cụ ghi nhật ký hiệu suất cao giúp xác định lỗi và các sự cố có thể xảy ra.
- Các đề xuất ghi nhật ký bao gồm:
  - Đăng nhập thường xuyên bằng các cấp độ khác nhau (gỡ lỗi, thông tin, lỗi).
  - Khi ghi nhật ký, hãy cung cấp thông tin theo ngữ cảnh dưới dạng đối tượng JSON.
  - Giám sát và lọc nhật ký bằng API truy vấn nhật ký (tích hợp sẵn cho nhiều trình ghi nhật ký) hoặc phần mềm xem nhật ký.
  - Hiển thị và sắp xếp các báo cáo nhật ký bằng các công cụ thông minh hoạt động như Splunk.

- Ví dụ mã

```
const pino = require('pino');

// your centralized logger object
const logger = pino();

// custom code somewhere using the logger
logger.info({ anything: 'This is metadata' },
    'Test Log Message with some parameter %s', 'some parameter');
```

Bài 4.

Xử lý lỗi và khắc phục sự cố

- 4.1. Các lỗi phổ biến nhất về cài đặt
- 4.2. Các lỗi phổ biến nhất về thông báo lỗi và lập trình
- 4.3. Sử dụng Async-Await và promises để async xử lý lỗi không đồng bộ
- 4.4. Chỉ sử dụng đối tượng Lỗi tích hợp
- 4.5. Phân biệt lỗi vận hành và lập trình viên
- 4.6. Xử lý lỗi tập trung
- 4.7. Sử dụng trình ghi nhật ký cập nhật để tăng khả năng hiển thị lỗi
- 4.8. Nắm bắt các chỉ lệnh promises chưa được xử lý
- 4.9. Thất bại nhanh chóng, xác thực các đối số bằng thư viện chuyên dụng

Từ chối Promise chưa được xử lý

- Nói chung, hầu hết mã ứng dụng Node.js/Express hiện đại đều chạy bên trong các lời hứa, cho dù bên trong trình xử lý .then, hàm callbacks hay catch blocks.
- Đáng ngạc nhiên là các lỗi được đưa ra ở những nơi này không được xử lý bởi trình xử lý sự kiện Ngoại lệ và biến mất, trừ khi nhà phát triển nhớ thêm mệnh đề .catch.
- Các phiên bản gần đây của Node đã thêm thông báo cảnh báo khi hiển thị từ chối chưa được xử lý. Mặc dù điều này có thể giúp bạn nhận thấy khi có sự cố xảy ra, nhưng rõ ràng đó không phải là phương pháp xử lý lỗi phù hợp.
- Một giải pháp đơn giản là chuyển hướng đến trình xử lý lỗi tập trung mà không quên thêm mệnh đề .catch trong mỗi lệnh gọi chuỗi promise.
- Việc xây dựng một chiến lược xử lý lỗi dựa trên các nguyên tắc của nhà phát triển là không chắc chắn.
- Cân nhắc sử dụng một phương án dự phòng và đăng ký process.on('unhandledRejection', callback). Điều này sẽ xử lý tất cả các lỗi đã được chỉ định nếu không được xử lý cục bộ.

- Ví dụ về mã: các lỗi này sẽ không bị bắt bởi bất kỳ trình xử lý lỗi nào (ngoại trừ unhandledRejection)

```
DAL.getUserById(1).then((johnSnow) => {
    // this error will just vanish
    if(johnSnow.isAlive === false)
        throw new Error('ahhhh');
});
```

- Ví dụ mã: Nắm bắt Promises chưa được giải quyết và bị từ chối

```
process.on('unhandledRejection', (reason, p) => {
    // I just caught an unhandled promise rejection,
    // since we already have fallback handler for
    // unhandled errors (see below),
    // let throw and let him handle that
    throw reason;
});

process.on('uncaughtException', (error) => {
    // I just received an error that was never handled,
    // time to handle it and then decide
    // whether a restart is needed
    errorManagement.handler.handleError(error);
    if (!errorManagement.handler.isTrustedError(error))
        process.exit(1);
});
```

Bài 4.

Xử lý lỗi và khắc phục sự cố

- 4.1. Các lỗi phổ biến nhất về cài đặt
- 4.2. Các lỗi phổ biến nhất về thông báo lỗi và lập trình
- 4.3. Sử dụng Async-Await và promises để async xử lý lỗi không đồng bộ
- 4.4. Chỉ sử dụng đối tượng Lỗi tích hợp
- 4.5. Phân biệt lỗi vận hành và lập trình viên
- 4.6. Xử lý lỗi tập trung
- 4.7. Sử dụng trình ghi nhật ký cập nhật để tăng khả năng hiển thị lỗi
- 4.8. Nắm bắt các chỉ lệnh promises chưa được xử lý
- 4.9. Thất bại nhanh chóng, xác thực các đối số bằng thư viện chuyên dụng

Để tránh các lỗi ẩn

- Tất cả chúng ta đều biết tầm quan trọng của việc kiểm tra các đối số và nhanh chóng thất bại để tránh các lỗi ẩn (xem ví dụ về mã anti-pattern bên dưới).
- Nếu không, hãy đọc về Lập trình rõ ràng và Lập trình phòng thủ.
- Trên thực tế, chúng ta có xu hướng tránh điều này vì những phiền toái khi mã hóa (ví dụ: nghĩ đến việc xác thực các đối tượng JSON phân cấp với các trường như email và ngày tháng) – Các thư viện như Joi và Trình xác thực biến nhiệm vụ tẻ nhạt này trở nên dễ dàng.
- Lập trình phòng thủ là một cách tiếp cận để cải thiện phần mềm và mã nguồn của nó về chất lượng chung, giảm số lỗi và sự cố phần mềm.
- Làm cho mã nguồn dễ hiểu – Mã nguồn phải dễ đọc và dễ hiểu để được kiểm tra trình phiên dịch.
- Nó đảm bảo rằng phần mềm hoạt động theo cách có thể dự đoán được bất chấp hành động của người dùng hoặc thông tin đưa vào ngoài dự kiến.

- Ví dụ về mã: xác thực đầu vào JSON phức tạp bằng cách sử dụng 'Joi'

```
var memberSchema = Joi.object().keys({
    password: Joi.string().regex(/^[a-zA-Z0-9]{3,30}$/),
    birthyear: Joi.number().integer().min(1900).max(2013),
    email: Joi.string().email()
});

function addNewMember(newMember) {
    // assertions come first
    Joi.assert(newMember, memberSchema);
    //throws if validation fails

    // other logic here
}
```

- Anti-pattern: không có xác nhận dẫn đến các lỗi khó chịu

```
// if the discount is positive
// let's then redirect the user to print his discount coupons
function redirectToPrintDiscount(httpResponse, member, discount) {
    if (discount != 0) {
        httpResponse.redirect(`/discountPrintView/${member.id}`);
    }
}

redirectToPrintDiscount(httpResponse, someMember);
// forgot to pass the parameter discount,
// why the heck was the user redirected to the discount screen?
```



Kết thúc tài liệu

SAMSUNG



ⓒ2022 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.